PROJECT REPORT

BSc COMPUTER SCIENCE

MUSLIM ASSOCIATION COLLEGE OF ARTS AND SCIENCE

Panavoor, Thiruvananthapuram, Kerala

(*Affiliated to the University of Kerala*)

AI-Powered Customer Segmentation System

By

ABDUL RAHEEM (32023893001)

ASHISH BINU (32023893007)

ADHIL MUHAMMED (32023893003)

SAFWAN YASIR (32023893021)

Submitted in partial fulfillment of

the requirements for the award of B.Sc (Computer Science) degree of

University of Kerala

2026

MUSLIM ASSOCIATION COLLEGE OF ARTS AND SCIENCE

Panavoor, Thiruvananthapuram, Kerala

(*Affiliated to the University of Kerala*)

Certificate

Certified that this is report titled "**AI-Powered Customer Segmentation System**" is a Bonafide record of the project work done by **ABDUL RAHEEM (32023893001), ASHISH BINU (32023893007), ADHIL MUHAMMAD (32023893003), SAFWAN YASIR (32023893021)** under our supervision and guidance, towards partial fulfillment of the requirements for the award of the Degree of B.Sc Computer Science of University of Kerala.

Head of the Department **Principal**

Internal Guide **External Examiner**

Date:

ACKNOWLEDGEMENT

Apart from the efforts of me, the success of this project depends largely on the encouragement and guidelines of many others. I take this opportunity to express my gratitude to the people who have been instrumental in the successful completion of this project.

I would like to show my greatest appreciation to **Mr. Rinu Raveendran.** I can't say thank you enough for his tremendous support and help. I feel motivated and encouraged every time I attend his meeting. Without his encouragement and guidance this project would not have materialized.

We are extremely tanking to our Head of the Department **Ms. Sumaiyah Mahjabin** for his valuable suggestions and encouragements. The guidance and support received from all the members who contributed and who are contributing to this project, was vital for the success of the project. I am grateful for their constant support and help.

Sincerely,

ABDUL RAHEEM (32023893001)

ASHISH BINU (32023893007)

ADHIL MUHAMMED (32023893003)

SAFWAN YASIR (32023893021)

ABSTRACT

The **AI-Powered Customer Segmentation System** is a production-grade machine learning application designed to segment retail customers using transactional invoice data. The system processes large-scale retail datasets (541,909+ records) and transforms raw transactional information into actionable customer intelligence through automated data cleaning, feature engineering, and unsupervised clustering techniques.The solution implements a modular, industry-ready architecture that separates training and inference pipelines. The training pipeline performs invoice-level data sanitization (removal of cancellations, handling missing customer identifiers, correction of negative quantities), computes derived transactional metrics, and aggregates customer-level features. Recency, Frequency, and Monetary (RFM) metrics are generated and standardized before applying the K-Means clustering algorithm to identify natural behavioral groupings within the customer base. Model artifacts—including the trained clustering model, feature scaler, and schema definitions—are versioned and persisted for reproducible inference.The inference pipeline ensures consistency by applying identical feature engineering transformations before generating cluster predictions. Post-processing logic maps raw cluster outputs into business-aligned segments such as High-Value Loyal Customers, At-Risk Customers, Recent Low Spenders, and Bulk Buyers.A Streamlit-based interactive dashboard enables administrative users to upload new datasets, execute segmentation workflows, visualize cluster distributions through dynamic analytics, and export enriched datasets for downstream business use. The system is designed for scalability, maintainability, and production deployment, providing organizations with data-driven insights that support targeted marketing, retention strategies, and revenue optimization initiatives.This project demonstrates the practical application of unsupervised learning in real-world retail analytics, bridging machine learning engineering principles with business decision-making frameworks.

CONTENTS

1. INTODUCTION. 1

1.1 ABOUT THE PROJECT 2

1.2 ORGANISATIONAL OVERVIEW 3

2. SYSTEM ANALYSIS AND DESIGN 4

2.1 EXISTING SYSTEM 5

2.1.1 Limitations of Existing System 5

2.2 PROPOSED SYSTEM 5

2.2.1 Advantages of Proposed System 5

2.2.2 Features of Proposed System 5

2.3 FEASIBILITY STUDY 6

2.3.1 Economic Feasibility 6

2.3.2 Technical Feasibility 6

2.3.3 Operational Feasibility 6

2.4 Hardware and Software Requirements 7

2.4.1 Hardware Requirements 7

2.4.2 Software Requirements 7

2.4.3 Functional Requirements 8

2.4.4Non-Functional Requirements 10

3. Project Description 14

3.1 FEASIBILITY STUDY 14

3.1.1 Economic Feasibility 14

3.1.2 Operational Feasibility… 15

3.1.3 Technical Feasibility… 15

4. SYSTEM DESIGN 25

4.1 INPUT DESIGN 28

4.2 OUTPUT DESIGN OF VISITOR 28

4.3 DATABASE DESIGN 32

4.3.1 Table Design… 32

4.3.2 Data Dictionary 34

4.4 MODELLING 36

4.4.1 Data Flow Diagram 36

4.4.2 Entity Relationship Diagram 31

4.5 ARCHITECTURAL DESIGN 40

4.5.1 Structure Chart 40

4.6 PROCEDURAL DESIGN. 42

4.6.1 Flow Chart 43

4.7 Menu Tree 42

5. SYSTEM TESTING AND IMPLEMENTATION 45

5.1 SYSTEM TESTING. 45

5.2 IMPLIMENTATION 39

6. SYSYTEM MAINTENANCE` 51

7. FUTURE ENHANCEMENT 53

8. CONCLUSION 55

9. BIBLIOGRAPHY 57

10.APPENDIX 58

10.1 System Coding 58

10.2 Screenshots 84

10.3 Meeting Minutes 92

10.4 Gantt Chart 94

INTRODUCTION

1.1 ABOUT THE PROJECT

The **AI-Powered Customer Segmentation System** is a production-grade, end-to-end machine learning solution designed to intelligently segment retail customers using large-scale transactional data. Built with Python, Scikit-Learn, and Streamlit, the system transforms raw invoice-level records into structured behavioral insights through automated data cleaning, feature engineering, and unsupervised clustering. Traditional customer analysis methods often rely on manual filtering, spreadsheet-based grouping, or static business rules, which are inefficient, inconsistent, and incapable of uncovering hidden behavioral patterns. This project eliminates those limitations by implementing a scalable and modular ML pipeline that operates with reproducibility, consistency, and business interpretability.

The system processes over 541,000 retail transaction records containing invoice details, product information, quantities, pricing, customer identifiers, and country data. Since real-world retail datasets contain inconsistencies such as cancelled invoices, negative quantities (returns), and missing customer identifiers, the application incorporates a robust data-cleaning layer before any modeling occurs. All cancellations are removed, invalid records are filtered, and date fields are converted into standardized datetime formats. Transaction-level revenue is computed through derived features, and customer-level aggregation is performed to prepare structured inputs for clustering.

At the core of the system is a behavior-driven segmentation engine powered by the K-Means clustering algorithm. The model operates on engineered customer-level features derived from transactional history. Before clustering, features are standardized using a persisted scaler to ensure consistent magnitude and distribution. The training pipeline stores all critical artifacts—including the trained clustering model, feature scaler, and feature schema—to guarantee reproducible inference and deployment stability. The system strictly separates training and inference workflows, ensuring that new incoming data follows the exact same transformation logic as the training dataset.

A major strength of the architecture is its production-oriented modular design. The project structure clearly separates components into pipeline modules (feature engineering and model training), inference modules (batch prediction logic), utilities (I/O handling), artifact storage, and a frontend deployment

layer. This separation ensures maintainability, scalability, and ease of future enhancements such as automated retraining or cloud deployment.

The deployment layer is implemented using Streamlit, providing an interactive administrative dashboard for business users. The interface allows CSV/Excel uploads, real-time data previews, execution of the segmentation pipeline, dynamic cluster visualization using Plotly, and downloadable enriched datasets. After prediction, raw cluster IDs are mapped to business-aligned segments such as High-Value Loyal Customers, At-Risk Customers, Recent Low Spenders, and Average/Bulk Buyers, ensuring that technical model outputs are translated into actionable business insights.

With artifact persistence, consistent feature schema enforcement, modular architecture, and an interactive visualization layer, the system delivers a scalable, maintainable, and deployment-ready customer intelligence platform suitable for real-world retail environments. It demonstrates how unsupervised machine learning can be engineered into a practical business solution that bridges advanced analytics with strategic decision-making.

1.2 ORGANIZATIONAL OVERVIEW

The **AI-Powered Customer Segmentation System** is a production-oriented machine learning solution developed to transform raw retail transaction data into structured, business-ready customer intelligence. Designed using Python, Scikit-Learn, and Streamlit, the system replaces traditional spreadsheet-based customer analysis with a scalable, automated segmentation framework capable of handling large transactional datasets exceeding 500,000 records. By leveraging unsupervised learning techniques and modular ML engineering principles, the platform enables organizations to transition from reactive decision-making to data-driven customer strategy formulation.

At the architectural core of the system is a clearly defined separation between the training pipeline and the inference pipeline. The training workflow performs structured data sanitization, feature derivation, customer-level aggregation, feature scaling, and K-Means clustering before persisting model artifacts for reproducibility. These artifacts—including the trained clustering model, scaler, and feature schema—ensure that any future inference strictly adheres to the same transformation logic as the training phase. This disciplined separation prevents training–serving skew and aligns the system with industry-grade ML deployment standards.

The intelligence of the platform lies in its ability to convert unstructured invoice-level transactions into mathematically structured customer representations. After cleaning inconsistencies such as cancelled invoices, negative quantities, and missing identifiers, the system computes derived transactional metrics and aggregates them into customer-level behavioral indicators. These engineered features are standardized and fed into a clustering algorithm that identifies natural groupings within the customer base. Rather than relying on predefined business rules, the system discovers patterns directly from behavioral data, ensuring unbiased and scalable segmentation.

Organizational reliability and maintainability are embedded into the project structure. The modular directory layout separates pipelines, inference logic, utility functions, artifact storage, and frontend

components. This design ensures that future enhancements—such as automated retraining, model replacement, cloud deployment, or performance monitoring—can be implemented without restructuring the entire system. By persisting artifacts and enforcing schema validation, the system guarantees consistency across environments and deployment cycles.

The deployment layer, built using Streamlit, provides a controlled administrative interface for business users. Through secure login access, administrators can upload CSV or Excel datasets, preview transactional records, execute segmentation, visualize cluster distributions using interactive analytics, and export enriched outputs. This interactive dashboard bridges technical machine learning outputs with operational business usability, ensuring that insights are not confined to data science teams but are accessible to decision-makers.

By combining robust data engineering practices, unsupervised machine learning, artifact-driven reproducibility, and an intuitive deployment interface, the AI-Powered Customer Segmentation System establishes a scalable, maintainable, and enterprise-aligned framework for behavioral customer analytics in real-world retail environments.

SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

In traditional retail environments, customer segmentation is typically performed using manual analysis, spreadsheet-based filtering, or static rule-driven categorization. Businesses often rely on basic metrics such as total sales or purchase frequency to identify “top customers,” without applying structured analytical modeling. These methods are largely reactive and depend on predefined assumptions rather than data-driven pattern discovery.Existing approaches do not provide a unified application layer that supports automated batch segmentation, visualization dashboards, and real-time customer scenario evaluation within a single deployable system.

Most retail organizations export transactional data into spreadsheets and manually group customers based on revenue thresholds or purchase counts. While this approach may provide a rough classification, it lacks mathematical rigor and does not account for multidimensional behavioral patterns. As transaction volumes increase—often reaching hundreds of thousands of records—manual segmentation becomes inefficient, error-prone, and non-scalable.

Some enterprises adopt basic Business Intelligence (BI) dashboards to visualize sales trends, but these tools often stop at descriptive analytics. They do not perform unsupervised clustering or behavioral grouping based on structured feature engineering. Additionally, segmentation rules are typically hard-coded, requiring manual updates whenever business definitions change.

Earlier analytical systems also failed to maintain reproducibility and consistency. Feature transformations were not standardized, training and inference logic were not separated, and models—if used—were not persisted as artifacts. This leads to inconsistencies between analysis cycles and makes deployment difficult in production environments

2.1.1 Limitations of Existing System

1. Manual Processing Overhead: Spreadsheet-based segmentation consumes significant analyst time and becomes impractical for large-scale datasets (500k+ records).

2. Static Rule-Based Segmentation: Fixed thresholds (e.g., revenue > X) fail to capture hidden behavioral patterns and do not adapt to evolving customer behavior.
3. Lack of Automation: No end-to-end pipeline for cleaning, feature engineering, and segmentation. Each step is performed manually or semi-manually.
4. Inconsistent Data Cleaning: Cancellations, returns, and missing customer identifiers are often not systematically handled, leading to biased analysis.
5. No Training–Inference Separation: Analytical scripts are typically ad hoc and do not separate model training from prediction workflows, causing reproducibility issues.
6. Absence of Artifact Management: Models and preprocessing steps are not versioned or persisted, leading to inconsistent results across runs.
7. Limited Scalability: Manual and rule-based systems struggle with large, continuously growing retail datasets.
8. No Real-Time or Batch Prediction Capability: Existing approaches do not support reusable prediction pipelines for new incoming data.
9. Poor Business Interpretability: Raw sales summaries do not translate directly into actionable customer segments aligned with retention or marketing strategies.
10. High Dependency on Technical Personnel: Non-technical stakeholders cannot independently run segmentation or download insights without analyst intervention.
11. Lack of Interactive Simulation Capability:Existing solutions do not provide individual customer scenario testing, preventing businesses from evaluating how changes in behavioral metrics impact customer classification

2.2 PROPOSED SYSTEM

The proposed **AI-Powered Customer Segmentation System** is a production-ready machine learning solution designed to automate behavioral customer segmentation using retail transactional data. Unlike traditional spreadsheet-based or rule-driven approaches, this system implements a structured ML pipeline that performs automated data cleaning, feature engineering, clustering, and business-segment mapping within a scalable and reproducible architecture.

The system processes raw invoice-level retail data and transforms it into customer-level behavioral representations through a controlled feature engineering workflow. It eliminates inconsistencies such as cancelled transactions, negative quantities (returns), and missing customer identifiers before aggregation. After deriving transactional metrics, the system computes standardized customer features and applies the K-Means clustering algorithm to identify natural behavioral groupings.A key strength of the proposed system is its strict separation between the training pipeline and inference pipeline. During training, the model is fitted on engineered and scaled customer data, and all artifacts—including the trained clustering model, feature scaler, and feature schema—are persisted. During inference, new datasets are processed using the exact same transformation logic, ensuring consistency and eliminating training–serving skew. This makes the system deployment-ready and suitable for real-world retail analytics.The deployment layer, implemented using Streamlit, provides an interactive administrative dashboard where users can upload CSV/Excel datasets, execute segmentation, visualize cluster distributions, and download enriched customer insights. Post-clustering, raw cluster IDs are mapped to business-aligned categories such as High-Value Loyal Customers, At-Risk Customers, Recent Low Spenders, and Average/Bulk Buyers, ensuring that machine learning outputs are directly actionable.The proposed system includes an interactive Customer Scenario Simulator that allows administrators to input behavioral metrics and instantly evaluate the predicted customer segment.This enables strategic decision testing by simulating how changes in recency, frequency, spending, and purchasing diversity impact customer classification.

2.2.1 Advantages of Proposed System

i.

2. Automated Segmentation: Customer groups are generated automatically using unsupervised machine learning without manual threshold setting.

3. Scalable Architecture: Efficiently processes large-scale datasets (500k+ transactions) without performance degradation.
4. Consistent Data Cleaning: Systematically removes cancellations and invalid records to ensure unbiased modeling.
5. Reproducible Pipelines: Separate training and inference workflows ensure stable and consistent predictions.
6. Artifact Persistence: Model, scaler, and schema files are stored for reliable future predictions and deployment.
7. Business-Aligned Outputs: Technical cluster labels are mapped to interpretable business segments.
8. Interactive Dashboard: Enables non-technical stakeholders to upload data, run segmentation, and export results independently.
9. Reduced Human Error: Eliminates manual grouping mistakes common in spreadsheet-based segmentation.
10. Real-Time Batch Processing Capability: Supports segmentation of new incoming datasets using the inference pipeline.
11. Deployment-Ready Design: Modular architecture allows easy extension, retraining, and integration with enterprise systems.
12. Scenario-Based Decision Support:Enables individual customer simulation to test strategic impact before launching marketing or retention campaigns

2.2.2 Features of Proposed System

The proposed system incorporates a structured feature engineering module that transforms invoice-level transactional data into customer-level analytical features. It ensures data consistency by enforcing schema validation and applying identical preprocessing steps during both training and inference phases.

The clustering module uses a standardized feature space to perform unsupervised learning, identifying hidden behavioral patterns within the customer base. The system persists all artifacts required for reproducible prediction, enabling stable deployment across environments.

The application includes a user-friendly Streamlit interface that allows administrators to securely upload datasets, preview records, execute segmentation workflows, visualize cluster distributions using interactive analytics, and download enriched CSV outputs. The system maintains organized artifact storage and modular code separation, ensuring maintainability and future scalability.

Additionally, the architecture supports future enhancements such as automated retraining, model versioning, cloud deployment, performance monitoring, and integration with CRM or marketing automation platforms, making it a robust and extensible customer intelligence framework.

The proposed system incorporates an interactive Customer Scenario Simulator that enables administrators to input key behavioral metrics and instantly obtain segment predictions.This feature supports proactive decision-making by allowing businesses to evaluate strategic interventions before applying them to the broader customer base.

2.3 REQUIREMENT SPECIFICATION

2.2.3 HARDWARE REQUIREMENTS

Processor :Intel Core i3 / AMD Ryzen 3 or equivalent

Memory :4 GB minimum (8 GB preferred)

Storage :5 GB free disk space

Architecture :64-bit system

Internet :Required only for initial dependency installation

2.2.4 SOFTWARE REQUIREMENTS

Operating System :Windows 10 / Windows 11 /Ubuntu

Language :Python 3.9 or 3.10

ML Framework :Scikit-Learn

Data Processing :Pandas,

NumPy Visualization:Plotly

Web Framework :Streamlit

Persistence :Joblib

File Handling :OpenPyXL / XLrd (for Excel support

Environment :Conda / venv (Virtual Environment)

Version Control :Git (optional)

2.2.5 FUNCTIONAL REQUIREMENTS

Functional requirements define the specific operations the AI-Powered Customer Segmentation System must perform to ensure accurate, consistent, and automated customer segmentation. The system is divided into structured modules aligned with the training pipeline, inference pipeline, and deployment layer.

DATA INPUT MODULE

This module handles dataset ingestion for both training and inference workflows.

FN 1: Upload Transactional Dataset

This function allows administrators to upload retail transactional data.
Input: CSV or Excel file containing invoice-level retail data.
Output: Loaded dataset validated against required schema.

FN 2: Validate Dataset Schema

This function verifies that uploaded data contains required columns and correct data types.
Input: Uploaded dataset.
Output: Schema validation confirmation or structured error message.

DATA CLEANING MODULE

This module ensures transactional data integrity before feature engineering.

FN 3: Remove Cancelled Transactions

This function filters invoices marked as cancellations.
Input: Raw transactional dataset.
Output: Dataset excluding cancelled invoices.

FN 4: Handle Missing Customer Identifiers

This function removes records with null or invalid CustomerID values.

Input: Cleaned dataset.

Output: Dataset with valid customer identifiers only.

FN 5: Generate Transactional Revenue

This function computes derived transactional revenue.

Input: Quantity and UnitPrice columns.

Output: TotalPrice feature added to dataset.

FEATURE ENGINEERING MODULE

This module transforms invoice-level data into structured customer-level features.

FN 6: Convert Invoice Date to Datetime

This function standardizes date formats for time-based calculations.

Input: InvoiceDate column.

Output: Datetime-formatted column.

FN 7: Aggregate Customer-Level Features

This function groups transactional records by CustomerID.

Input: Cleaned transactional dataset.

Output: Customer-level aggregated dataset.

FN 8: Compute Behavioral Features

This function calculates customer-level behavioral metrics for clustering.

Input: Aggregated customer dataset.

Output: Structured feature matrix for modeling.

MODEL TRAINING MODULE

This module performs clustering and artifact persistence.

FN 9: Scale Features

This function standardizes features using a trained scaler.

Input: Customer feature matrix.

Output: Scaled feature matrix.

FN 10: Train Clustering Model

This function applies K-Means clustering to grouped customer data.

Input: Scaled feature matrix.

Output: Cluster assignments and trained model.

FN 11: Persist Model Artifacts

This function saves trained artifacts for reproducibility.

Input: Trained model, scaler, and feature schema.

Output: Stored `.pkl` and `.json` files in artifacts directory.

<u>INFERENCE MODULE</u>

This module applies trained artifacts to new incoming data.

FN 12: Load Persisted Artifacts

This function loads saved model, scaler, and schema for prediction.

Input: Stored artifact files.

Output: Ready-to-use model components.

FN 13: Apply Consistent Feature Engineering

This function ensures inference data follows identical transformation steps as training.

Input: New transactional dataset.

Output: Scaled feature matrix aligned with training schema.

FN 14: Predict Customer Cluster

This function assigns cluster labels to customers.

Input: Scaled feature matrix.

Output: Cluster ID for each customer.

FN 15: Map Cluster to Business Segment

This function translates raw cluster IDs into interpretable business categories.

Input: Predicted cluster ID.

Output: Business segment label (e.g., High-Value Loyal, At-Risk, etc.).

DASHBOARD & OUTPUT MODULE

This module provides visualization and export capabilities.

FN 16: Display Segmentation Results

This function presents cluster distributions and insights in the dashboard.

Input: Segmented customer dataset.

Output: Interactive visualizations and summary metrics.

FN 17: Enable Data Download

This function allows users to export segmented results.

Input: Final segmented dataset.

Output: Downloadable CSV file with cluster and segment labels.

2.2.6 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality attributes and operational standards of the AI-Powered Customer Segmentation System. These requirements focus on system performance, scalability, reliability, security, and maintainability to ensure the solution operates as a production-grade ML application rather than a prototype script.

1. **Performance:**

Response Time: The system should process and segment a dataset of up to 500,000+ transactional records within an acceptable time frame (preferably under 30–60 seconds depending on hardware configuration).

Efficient Data Aggregation:

Customer-level feature aggregation and transformation should be optimized using vectorized operations (Pandas/Numpy) to minimize processing latency.

Model Inference Speed:

Cluster prediction during inference should execute in near real-time once features are engineered and scaled.

Optimized Artifact Loading:

Persisted artifacts (model, scaler, schema) should load efficiently at runtime without repeated reinitialization.

2. **Security:**

Data Confidentiality:

Customer transactional data must be handled securely and not exposed outside the system environment.

Access Control:

Only authorized administrators should be allowed to upload datasets, execute segmentation, and download results.

Artifact Protection:

Stored model artifacts (`.pkl`, `.json`) must be protected from unauthorized modification.

Secure File Handling:

Uploaded files must be validated before processing to prevent malicious input or schema corruption.

3. **Usability:**

User-Friendly Interface:

The Streamlit dashboard should provide a clean, intuitive interface suitable for non-technical business users.

Clear Workflow Navigation:

Users should easily navigate between data upload, preview, segmentation execution, visualization, and download sections.

Visual Interpretability:

Cluster distributions and segment summaries should be displayed using interactive and readable visualizations.

Error Handling:

The system should display structured error messages for missing columns, invalid data types, or failed uploads.

4. **Reliability:**

System Availability:

The system should remain stable during repeated segmentation operations without crashes or memory leaks.

Fault Tolerance:

The application should gracefully handle corrupted files, missing data, or partial uploads without terminating execution.

Data Integrity:

Segmentation results must remain consistent across multiple runs when using the same model artifacts.

Reproducibility:

Separation of training and inference pipelines must prevent training–serving skew and ensure consistent outputs.

5. **Accuracy & Consistency:**

Feature Consistency:

The inference pipeline must apply identical preprocessing logic as the training pipeline.

Model Stability:

Clustering results should remain stable across runs when using fixed random states.

Schema Enforcement:

Feature schema validation should ensure that prediction inputs match training configuration.

6. **Scalability:**

Dataset Scalability:

The system should handle increasing transaction volumes without architectural changes.

User Scalability:

Multiple segmentation sessions should be supported without reconfiguring the application.

Future Extensibility:

The modular architecture should allow integration of:

- Automated retraining pipelines
- Advanced clustering algorithms
- Database-backed storage
- Cloud deployment environments

2.3 FEASIBILITY STUDY

The feasibility study evaluates whether the proposed **AI-Powered Customer Segmentation System** is practical, sustainable, and valuable for retail organizations. It assesses the project from economic, operational, technical, and social perspectives to determine its viability for real-world deployment.

1. Economic Feasibility

2. Operational Feasibility

3. Technical Feasibility

4. Organizational Feasibility

2.3.1 ECONOMIC FEASIBILITY

Economic feasibility examines whether the system delivers measurable business value relative to its development and operational costs.

Development Cost:
The system is built using open-source technologies such as Python, Pandas, NumPy, Scikit-Learn, Plotly, and Streamlit. This eliminates licensing costs associated with proprietary analytics software.

Infrastructure Cost:
The solution runs efficiently on standard CPU-based systems without requiring GPU acceleration or specialized hardware. It can also be deployed on low-cost cloud virtual machines if needed.

Maintenance Cost:
Maintenance primarily involves model monitoring, periodic retraining, and dependency updates. Since the architecture is modular and artifact-driven, maintenance overhead is controlled and predictable.

Operational Savings:
The system reduces manual analytical effort previously required for spreadsheet-based segmentation. Marketing and retention strategies become data-driven, reducing resource wastage.

Return on Investment (ROI):
By enabling identification of high-value customers, at-risk segments, and spending patterns, the

system supports targeted campaigns and improved customer retention—leading to increased revenue potential.

Given its low implementation cost and high strategic value, the system is economically feasible.

2.3.2 OPERATIONAL FEASIBILITY

Operational feasibility evaluates how effectively the system integrates into business workflows.

Workflow Integration:
The system integrates into existing retail analytics processes by accepting transactional data exports (CSV/Excel) without requiring infrastructure redesign.

User Accessibility:
The Streamlit-based dashboard enables non-technical stakeholders to upload datasets, execute segmentation, and download results without data science expertise.

Automation Level:
The end-to-end pipeline automates data cleaning, feature engineering, scaling, clustering, and segment mapping, minimizing manual intervention.

Training Requirements:
Minimal user training is required. Administrators only need to understand data upload and interpretation of business segments.

Management Support:
Since the system directly supports revenue growth and customer retention strategies, it aligns well with business management objectives.

As it enhances existing analytics workflows without operational disruption, the project is operationally feasible.

2.3.3 TECHNICAL FEASIBILITY

Technical feasibility determines whether the required technology, tools, and expertise are available.

Technology Availability:

The system uses mature, industry-standard libraries such as Scikit-Learn for clustering and Streamlit for deployment.

Technical Expertise:

Development requires knowledge of Python, data preprocessing, and machine learning fundamentals—skills widely available in modern analytics teams.

Platform Compatibility:

The system runs on Windows, Linux, or macOS environments and supports both local and cloud deployment.

Architecture Stability:

The modular structure ensures separation between training and inference pipelines, preventing inconsistencies and improving maintainability.

Scalability:

The architecture supports increasing transactional volumes and can be extended to distributed processing frameworks if needed.

Integration Capability:

Future integration with CRM systems, marketing automation platforms, or enterprise databases is possible due to modular design and artifact persistence.

Since all required technologies and expertise are readily available, the project is technically feasible.

2.3.4 ORGANISATIONAL FEASIBILITY

Organizational feasibility evaluates stakeholder acceptance and long-term sustainability.

Business Alignment:

The system directly supports marketing, customer retention, and strategic decision-making objectives.

Data Governance Compatibility:

The application operates on structured transactional data and can comply with organizational data protection policies.

Decision-Making Transparency:

Business segment mapping ensures that model outputs are interpretable and actionable for management teams.

Strategic Advantage:

Data-driven segmentation enhances competitive positioning by enabling personalized marketing and optimized resource allocation.

With proper stakeholder alignment and governance policies, the system is organizationally sustainable and strategically beneficial.

SYSTEM DESIGN

3.1 DESIGN OF SUBSYSTEM

The subsystem design defines the structured organization of functional components within the AI-Powered Customer Segmentation System. Each subsystem is modular, responsibility-driven, and interacts through clearly defined interfaces. Core machine learning processes such as feature engineering, scaling, and clustering are abstracted from the user interface layer, ensuring clean separation of concerns and maintainability.

The AI-Powered Customer Segmentation System is divided into the following major subsystems:

- User Interface Subsystem
- Data Ingestion & Validation Subsystem
- Data Processing & Feature Engineering Subsystem
- Model Training Subsystem
- Inference & Segmentation Subsystem
- Artifact & Data Storage Subsystem

1. User Interface Subsystem

This subsystem is developed using Streamlit and acts as the interaction layer between business users and the ML engine.

Navigation Control:
Provides structured workflow navigation such as Data Upload, Data Preview, Run Segmentation, and Download Results.

File Upload Handling:
Accepts CSV and Excel transactional datasets.

Execution Control:
Triggers segmentation workflow (inference pipeline).

Visualization Display:

Displays cluster distributions, segment summaries, and customer-level insights using interactive charts.

Result Export:

Allows download of segmented customer data in CSV format.

Error Handling:

Displays structured validation messages for schema mismatch, missing columns, or corrupted files.

2. Data Ingestion & Validation Subsystem

This subsystem ensures the integrity and consistency of input datasets before processing.

Schema Validation:

Verifies presence of required columns (InvoiceNo, Quantity, UnitPrice, CustomerID, InvoiceDate, etc.).

Data Type Enforcement:

Ensures numeric and datetime fields are correctly formatted.

Missing Value Handling:

Removes records with missing customer identifiers.

Cancellation Filtering:

Excludes cancelled invoices and invalid transactions.

This subsystem guarantees that downstream modules operate on clean and structured data.

3. Data Processing & Feature Engineering Subsystem

This subsystem transforms transactional invoice-level data into customer-level analytical features.

Revenue Computation:

Generates derived transactional revenue feature.

Datetime Standardization:

Converts invoice date to datetime format for time-based calculations.

Customer Aggregation:

Groups data at the customer level.

Behavioral Feature Generation:

Computes structured behavioral indicators required for clustering.

Feature Scaling:

Applies standardized scaling using a persisted scaler to normalize feature distributions. This subsystem ensures consistency between training and inference pipelines.

4. Model Training Subsystem

This subsystem is responsible for clustering model development and artifact generation.

Clustering Model Initialization:

Configures K-Means with fixed parameters and random state for reproducibility.

Model Training:

Fits clustering model on scaled customer feature matrix.

Cluster Assignment:

Assigns cluster labels to customers in training dataset.

Artifact Persistence:

Stores trained model (`kmeans.pkl`), scaler (`scaler.pkl`), and schema (`feature_schema.json`).

This subsystem is executed separately from the inference workflow to maintain production-grade architecture.

5. Inference & Segmentation Subsystem

This subsystem applies trained artifacts to new incoming datasets.

Artifact Loading:

Loads persisted model, scaler, and feature schema.

Consistent Transformation:

Applies identical preprocessing logic as training pipeline.

Cluster Prediction:

Generates cluster labels for new customers.

Business Mapping:

Maps raw cluster IDs to interpretable business segments (e.g., High-Value Loyal, At-Risk, etc.).

This subsystem ensures reliable and repeatable segmentation for operational use.

6. Artifact & Data Storage Subsystem

This subsystem manages persistent storage of models and processed outputs.

Model Artifact Storage:

Maintains trained model and scaler files in the artifacts directory.

Schema Management:

Stores feature schema configuration to prevent feature mismatch.

Output Storage:

Handles segmented dataset exports.

Directory Organization:

Ensures modular separation of data, pipeline, inference, and utility components.

Data Integrity Assurance:

Prevents corruption of artifacts and enforces consistent file structure.

Subsystem Interaction Overview

- The User Interface Subsystem interacts with the Inference Subsystem to trigger segmentation.
- The Data Ingestion Subsystem feeds cleaned data into the Feature Engineering Subsystem.
- The Model Training Subsystem generates artifacts stored in the Artifact Subsystem.
- The Inference Subsystem retrieves artifacts and produces business-segmented outputs.
- The Artifact & Storage Subsystem maintains consistency across executions.

3.2 USER INTERFACE DESIGN

The User Interface (UI) of the AI-Powered Customer Segmentation System is developed using the Streamlit framework to provide a responsive, browser-based analytical dashboard that runs on the local host or deployment server. The UI is designed for business administrators and analysts who require a fast, intuitive, and data-driven interface for performing customer segmentation without interacting directly with the machine learning codebase.

The design emphasizes:

- Clear workflow navigation
- High-speed processing feedback
- Structured data upload and validation
- Interactive visual analytics
- Secure administrative access

The interface reflects the modular ML architecture, ensuring that the training artifacts and inference pipeline operate seamlessly behind the dashboard while maintaining real-time responsiveness.

3.2.1 INPUT DESIGN

The input design of the AI-Powered Customer Segmentation System is structured to ensure reliable data ingestion, strict validation, and accurate segmentation outcomes. The system primarily accepts input through structured file uploads and controlled UI actions.

1. Administrative Access Input

The system begins with an Admin Login interface to restrict access to authorized personnel. The login form captures:

- Username
- Password

Basic authentication validation ensures controlled system usage and prevents unauthorized segmentation operations.

2. Data Upload Interface

The primary input to the system is transactional retail data uploaded in:

- CSV format
- Excel format (.xlsx)

The upload component is built using Streamlit's file uploader module. Once uploaded, the system:

- Reads the file into a Pandas DataFrame
- Displays a preview of the dataset
- Shows dataset dimensions (rows × columns)
- Performs schema validation

The system verifies that required columns are present:

- InvoiceNo
- StockCode
- Quantity
- InvoiceDate
- UnitPrice
- CustomerID
- Country

If any mandatory column is missing, the system displays a structured error message and prevents further processing.

3. Data Validation Mechanisms

To maintain data integrity and ensure reliable clustering results, multiple validation layers are implemented:

- Detection of missing CustomerIDs
- Identification and removal of cancellation invoices (InvoiceNo starting with ‘C’)
- Handling of negative quantities (returns)
- Conversion validation for InvoiceDate to datetime format
- Verification of numeric consistency in Quantity and

UnitPrice The system provides immediate feedback if:

- File format is incorrect
- Required fields are absent
- Data types are incompatible
- File is empty or corrupted

This prevents model execution on invalid datasets.

4. Segmentation Execution Trigger

Once validation is successful, the user can initiate segmentation using a clearly labeled action

button: “Run Customer Segmentation”

Upon execution:

- The inference pipeline loads saved artifacts (`scaler.pkl`, `kmeans.pkl`)
- Feature engineering logic is applied
- RFM features are generated
- Scaling is applied
- Cluster prediction is performed

- Business segment mapping is assigned

A progress indicator informs the user during processing to enhance transparency.

5. Output and Visualization Interface

After segmentation:

- The segmented dataset is displayed in tabular format
- Cluster distribution summary is shown
- Business segment labels are presented
- Interactive visualizations (Plotly charts) display:
    - Segment distribution
    - Monetary contribution by segment
    - Frequency comparison
    - Recency distribution

The system also provides a Download Segmented CSV button for exporting results.

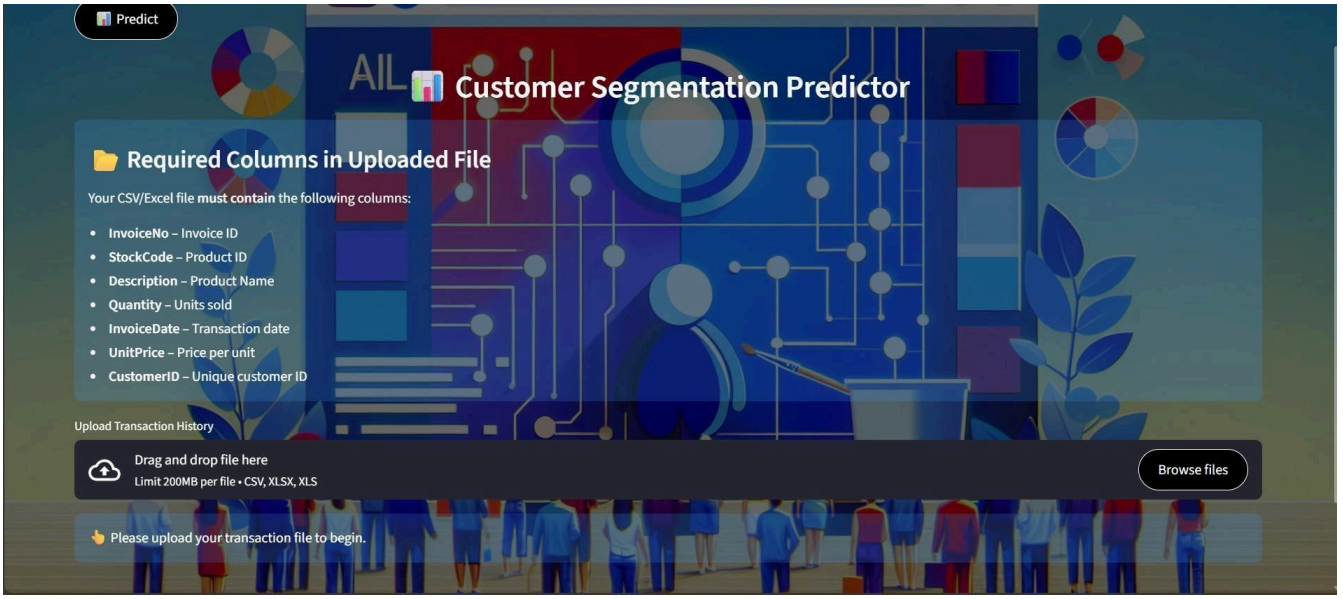
6. User Experience Considerations

The input design prioritizes:

- Simplicity for non-technical users
- Structured workflow (Login → Upload → Validate → Segment → Analyze → Download)
- Clear error handling
- Immediate visual feedback
- Minimal manual intervention

No manual feature entry is required, reducing human errors and ensuring reproducibility through automated pipeline execution.

AI-POWERED CUSTOMER SEGMENTATION SYSTEM DATA INPUT



3.2.2 OUTPUT DESIGN

The output design of the AI-Powered Customer Segmentation System focuses on analytical clarity, actionable business insights, and structured data presentation. The outputs are delivered through an interactive Streamlit dashboard, ensuring that business users can clearly interpret segmentation results and make data-driven decisions.

The design emphasizes:

- Real-time processing feedback
- Clear cluster distribution visibility
- Business-segment interpretation
- Downloadable structured reports
- Interactive visual analytics

1. Segmentation Results Display

Once the segmentation process is executed, the system displays the processed customer-level dataset in a structured tabular format. The output table includes:

- CustomerID
- Recency
- Frequency
- Monetary
- Assigned Cluster
- Business Segment Label

This structured presentation allows administrators to review clustering assignments and verify segmentation consistency.

A summary section displays:

- Total number of customers segmented
- Number of clusters formed
- Distribution count per cluster

This provides immediate insight into customer distribution across segments.

2. Business Segment Mapping Output

Raw cluster numbers generated by the K-Means model are mapped to business-friendly segment labels such as:

- High-Value Loyal Customers
- At-Risk Customers
- Recent Low Spenders
- Average/Bulk Buyers

This mapping layer ensures that technical clustering outputs are translated into strategic business categories. The system clearly displays both the cluster ID and its corresponding business label to maintain transparency.

3. Visual Analytics Dashboard

To enhance interpretability, the system integrates interactive Plotly visualizations, including:

- Cluster distribution bar chart
- Monetary contribution by segment
- Frequency comparison across segments
- Recency distribution trends

These charts allow users to visually analyze spending behavior patterns and identify high-performing or declining customer groups. Hover tooltips provide additional contextual details.

4. Data Export Functionality

The system provides a **Download Segmented CSV** option that enables users to export the enriched dataset containing.

- RFM features
- Cluster assignments
- Business segment labels

This allows integration with CRM systems, marketing platforms, or further BI analysis tools.

5. Status and Feedback Messages

The system displays structured output messages to ensure clarity:

- Success confirmation after segmentation
- Warning messages if dataset is empty after cleaning
- Error alerts if artifacts fail to load
- Informational messages during model execution

A progress indicator ensures users understand when the inference pipeline is running.

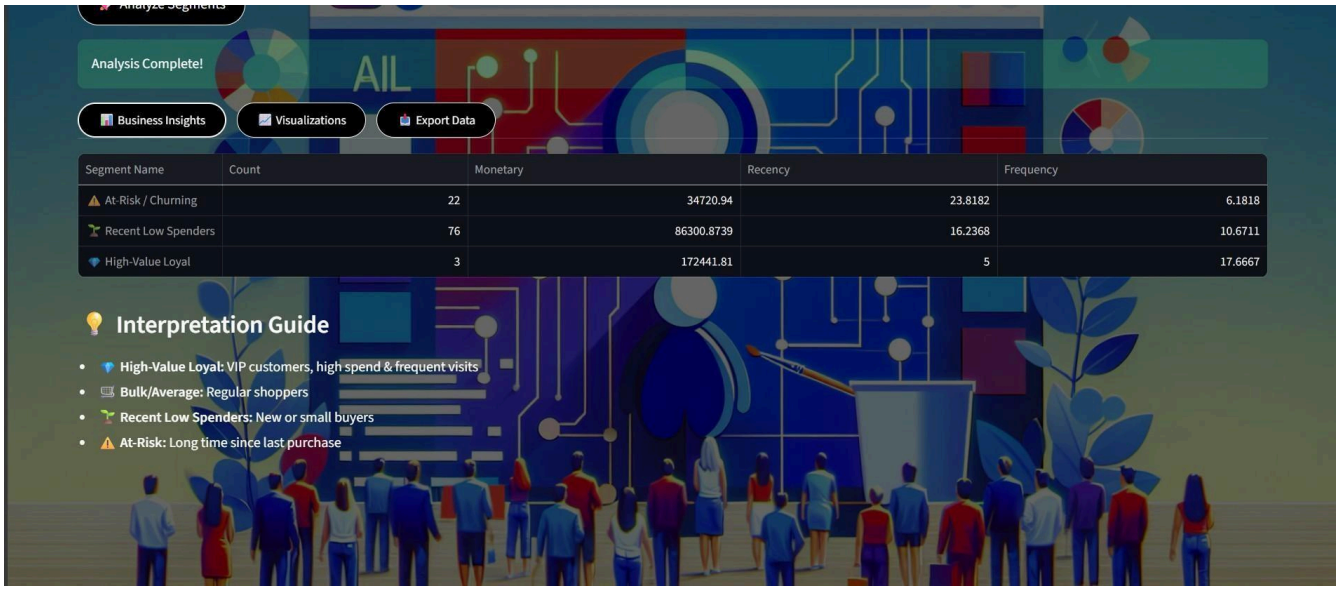
6. Business Insight Summary

In addition to raw data, the system provides concise analytical summaries such as:

- Segment contributing highest revenue
- Segment with lowest purchase frequency
- Segment with highest recency (potential churn risk)

This bridges the gap between machine learning output and executive-level decision-making.

SEGMENTED OUTPUT



Segment Name	Count	Monetary	Recency	Frequency
At-Risk / Churning	22	34720.94	23.8182	6.1818
Recent Low Spenders	76	86300.8739	16.2368	10.6711
High-Value Loyal	3	172441.81	5	17.6667

3.3 DATABASE DESIGN

The AI-Powered Customer Segmentation System does not use a traditional relational database such as MySQL or PostgreSQL. Instead, it follows a lightweight, file-based storage architecture optimized for machine learning workflows and analytical processing.

The system operates primarily using:

- In-memory data processing (Pandas DataFrames)
- Serialized ML artifacts (.pkl files)
- Structured schema configuration (JSON)
- User-uploaded CSV/Excel files
- Exportable segmented CSV outputs

This design is aligned with ML inference-based applications where persistent transactional storage is not required.

3.3.1 Artifact-Based Storage Architecture

The system uses a dedicated `artifacts/` directory to store trained components required for inference:

Stored Artifacts:

- `scaler.pkl` → StandardScaler object used during training
- `kmeans.pkl` → Trained K-Means clustering model
- `feature_schema.json` → Defines expected feature

structure These artifacts ensure:

- Reproducibility
- Consistent scaling during inference
- Decoupling of training and prediction pipelines
- Production-ready deployment.

The artifacts act as the persistent intelligence layer of the system.

3.3.2 Data Handling Workflow

Instead of storing data permanently, the system follows a dynamic processing model:

1. User uploads transactional dataset (CSV/Excel)
2. Data is loaded into a Pandas DataFrame
3. Cleaning and feature engineering is performed
4. RFM features are computed
5. Scaling and clustering are applied
6. Results are displayed and optionally exported

No transactional records are permanently stored in

the system. This ensures:

- Data privacy
- No long-term storage liabilities
- Stateless application behavior
- Simplified deployment

3.3.3 File-Based Output Storage

The system allows exporting segmented results as:

- CSV files containing:
    - CustomerID
    - RFM features
    - Cluster label
    - Business segment

mapping These files can be

integrated externally into:

- CRM systems
- Business Intelligence tools
- Marketing automation platforms

3.3.4 Data Integrity Mechanisms

Although no database constraints exist, integrity is enforced through:

- Schema validation (feature_schema.json)
- Column presence validation
- Data type validation
- Mandatory field checks
- Controlled feature engineering pipeline
- Consistent artifact loading

This replaces traditional database constraints such as:

- Primary Keys
- Foreign Keys
- CHECK constraints

The system instead relies on programmatic validation within the ML pipeline.

Architectural Justification

The absence of a database is intentional and appropriate because:

- The system performs analytical segmentation, not transactional management.
- It is inference-driven, not CRUD-driven.
- The ML model artifacts act as the persistent intelligence layer.
- Uploaded datasets are temporary analytical inputs.

This approach improves:

- Deployment simplicity
- Performance
- Portability
- Maintainability

3.3.1 TABLE DESIGN

Table :Raw Transaction Data Structure

Field Name	**Data Type**	**Description**
InvoiceNo	Object	Unique invoice identifier
StockCode	Object	Product code
Description	Object	Product description
Quantity	Integer	Units purchased
InvoiceDate	Datetime	Transaction timestamp
UnitPrice	Float	Price per unit
CustomerID	Float	Unique customer identifier
Country	Object	Customer country

Table : Customer-Level Feature Structure

Feature Name	**Data Type**	**Description**
CustomerID	Integer	Unique customer identifier
Recency	Integer	Days since last purchase
Frequency	Integer	Total no. of purchases
Monetary	Float	Total spending amount

3.3.2 Segmentation Output Structure

Column Name	Data Type	Description
CustomerID	Integer	Unique identifier
Recency	Integer	RFM feature
Frequency	Integer	RFM feature
Monetary	Float	RFM feature
Cluster_Label	Integer	K-Means assigned cluster
Business_Segment	String	Interpreted business category

3.4 MODELING

3.4.1 DATA FLOW DIAGRAM (DFD)

Data flow diagrams were proposed by Larry Constantine. It is a graphical representation of the flow of data through an information system, modeling its process aspects. A DFD is often used as a preliminary step to create an overview of the system, which can later be elaborated. DFDs can also be used for the visualization of data processing.

A DFD shows what kind of information will be input to and output from the system, where the data will come from and go to, and where the data will be stored. It does not show information about the timing of process or information about whether processes will operate in sequence or in parallel.

DATA FLOW NOTATIONS

External Entities

This represents any outside agency, which interact with the system. It represents the source or destination of data for the system under the consideration.

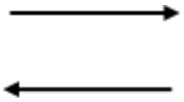
Process/Function

A process represents activities in which data is manipulated by being stored or retrieved or transferred in some way. The process transforms the input data into output data.

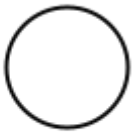
Data Flow Diagram Symbols

- Source or Destination of data

- Data Flow

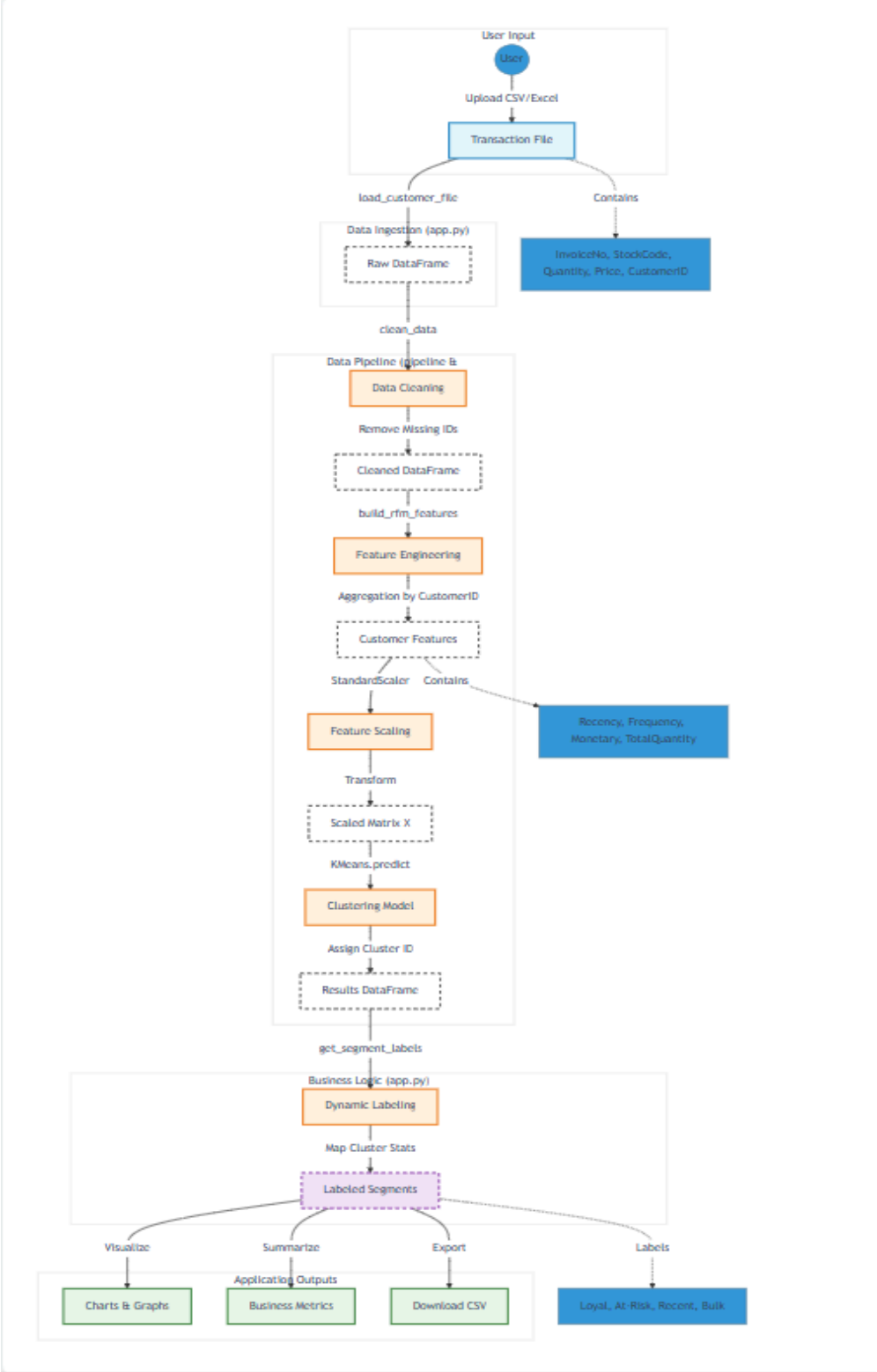


- Process

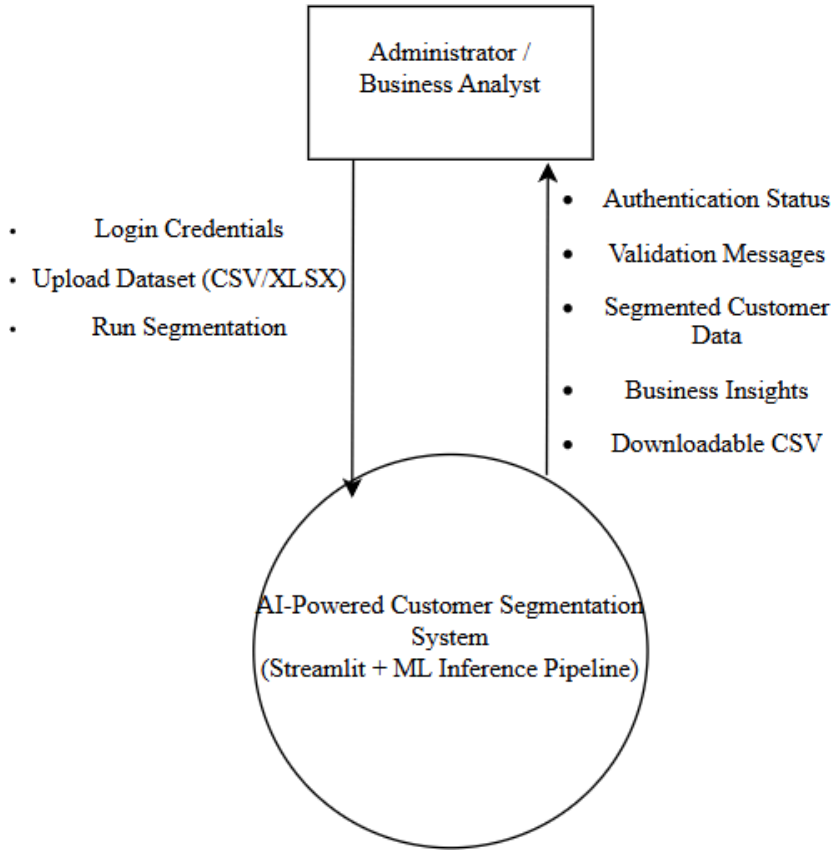


- Storage

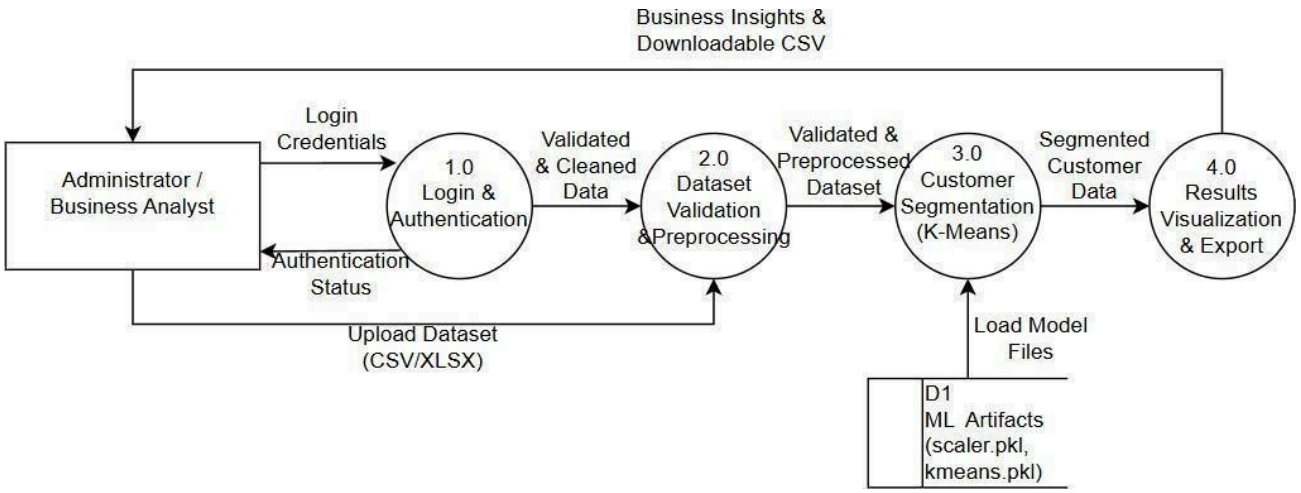
End-to-End Customer Segmentation Data Flow Diagram



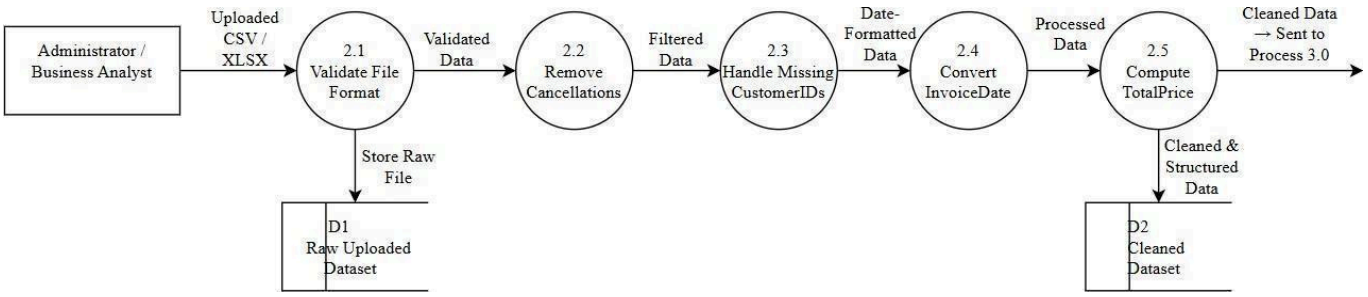
LEVEL 0/CONTEXT LEVEL



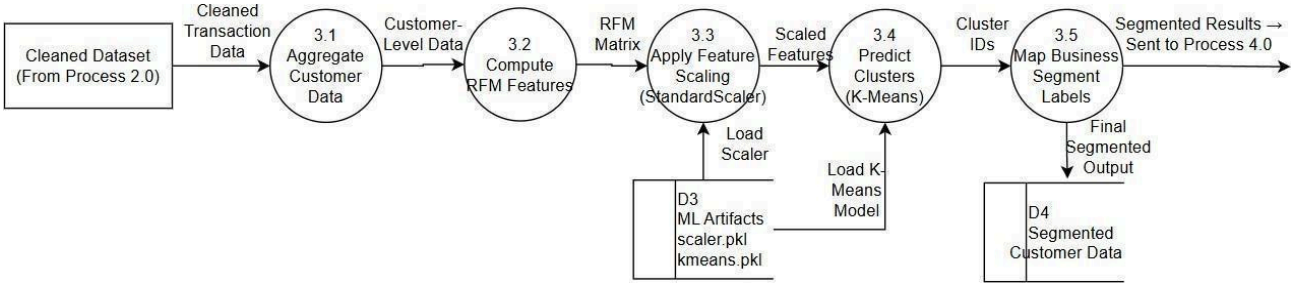
LEVEL 1



Level 2 DFD – Decomposition of 2.0 Dataset Validation & Preprocessing



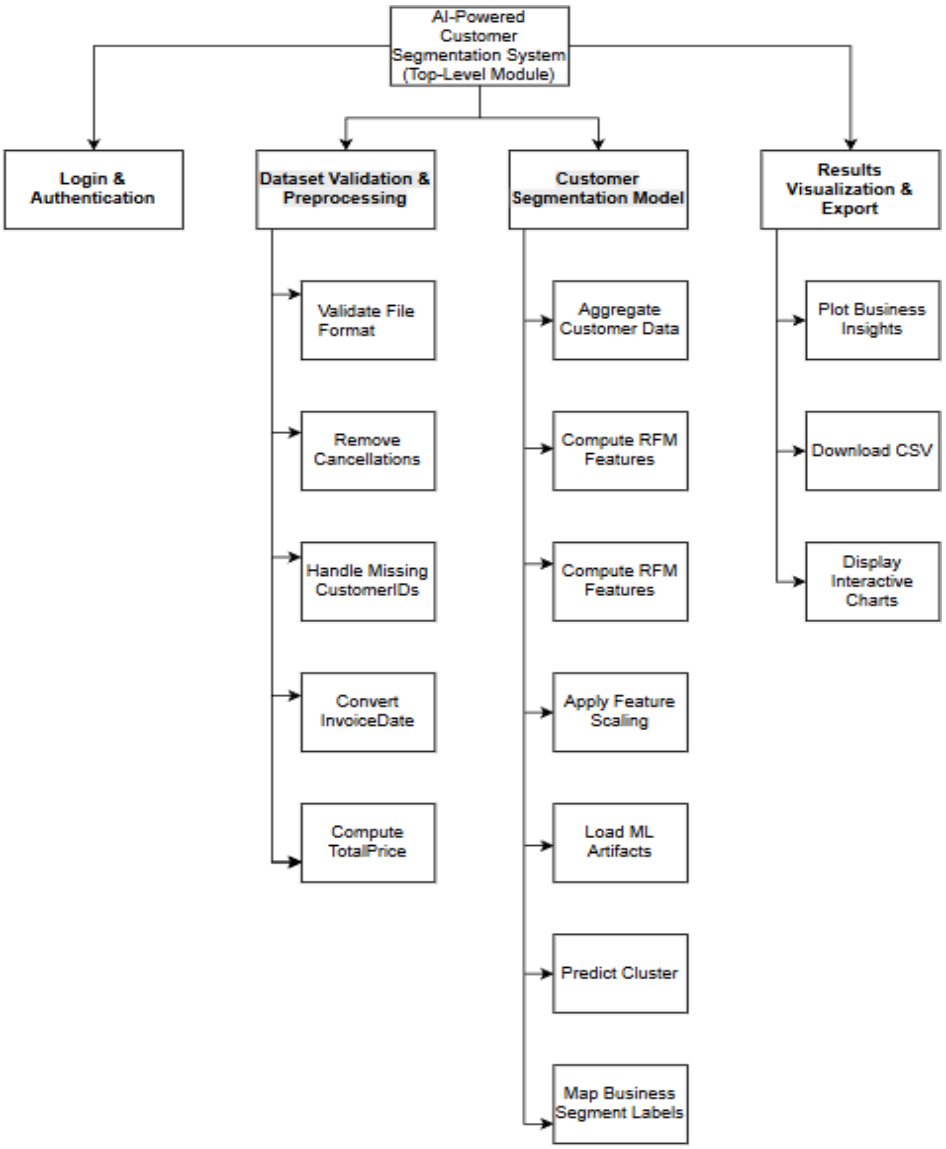
Level 2 DFD – Decomposition of 3.0 Customer Segmentation (K-Means)



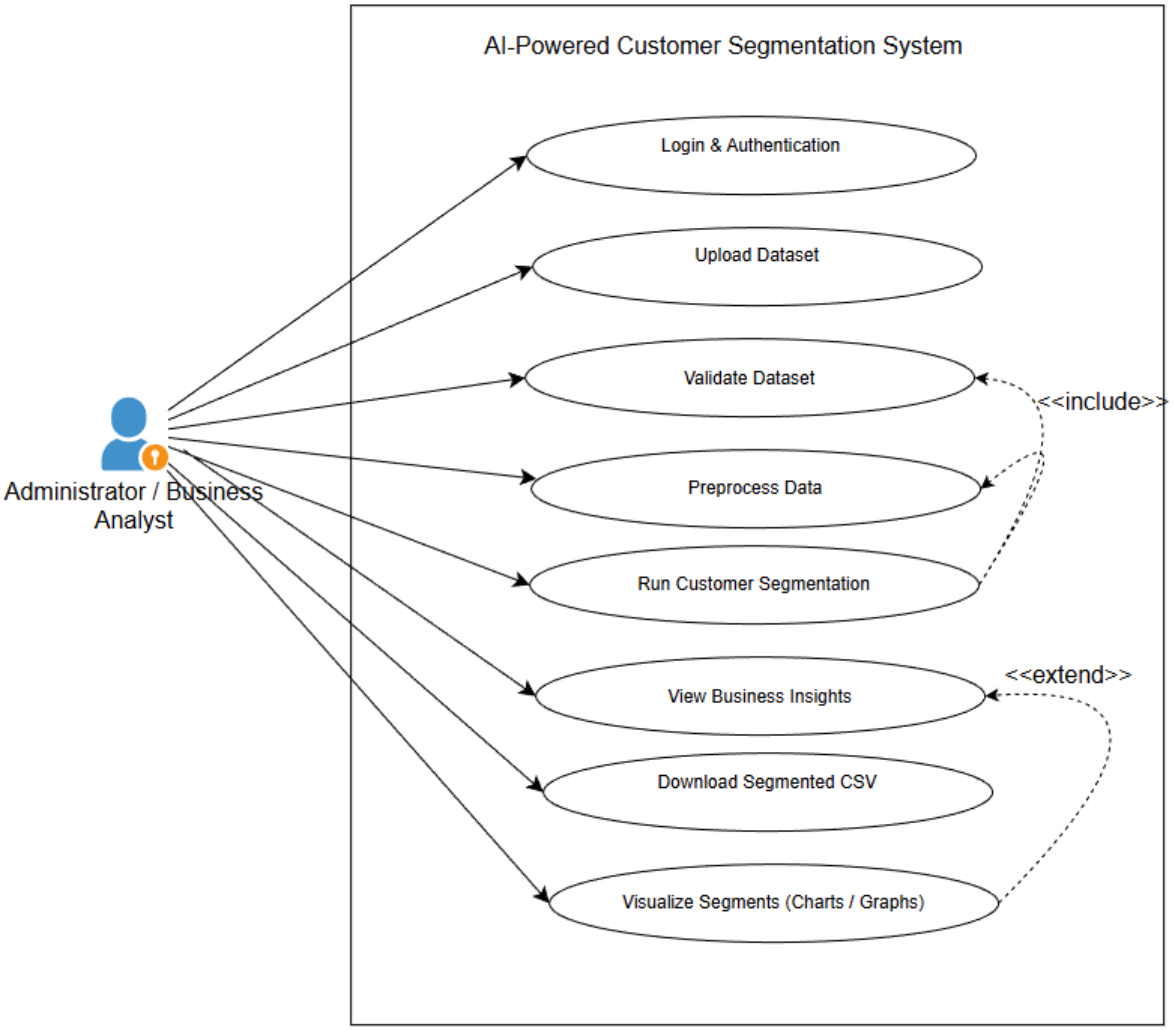
3.5 ARCHITECTURAL DESIGN

3.5.1 STRUCTURE CHART

A structure chart in software engineering is a chart which shows the breakdown of a system to its lowest manageable parts.They are used in structured programming to arrange program modules into a tree. Structure charts are an example of a top-down design where a problem (the program) is broken into its components.The system is divided into several components,each of which functions separately. Structure chart represent the functions and sub-functions of each of the modules.

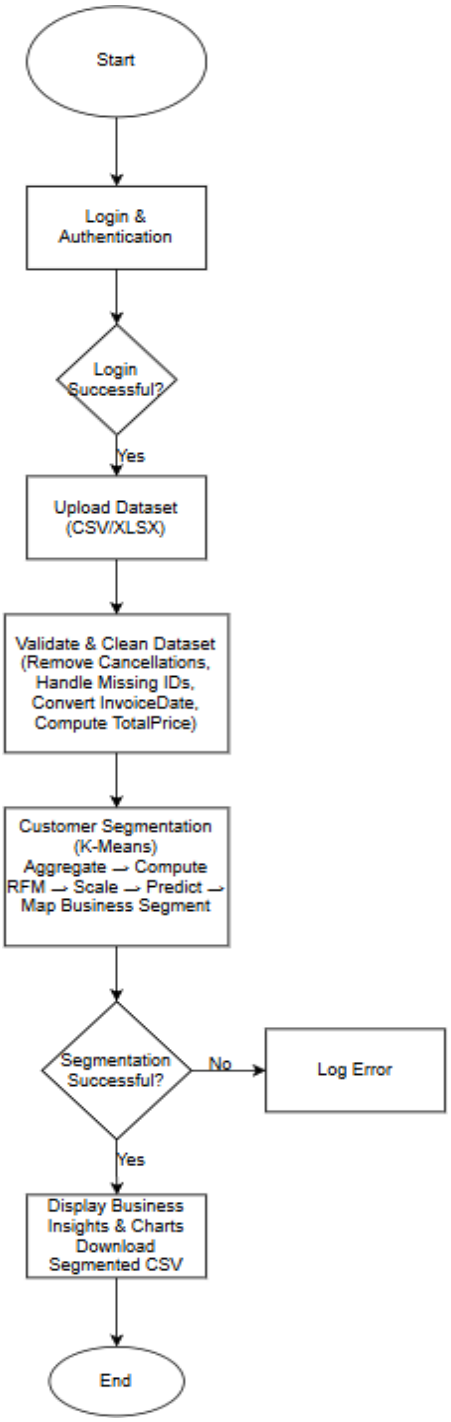


3.6 USE CASE DIAGRAM



3.7 PROCEDURAL DESIGN

3.7.1 FLOW CHART



PROCESSING ENVIRONMENT

4.1 HARDWARE SPECIFICATIONS

1. Processing Unit (CPU & GPU)

- **Processor (CPU):** Intel Core i5/i7 (8th Gen or higher) or AMD Ryzen 5/7 series recommended to handle large-scale data processing, RFM computations, and Streamlit dashboard rendering.
- **Multi-threading:** Minimum 4 cores required for concurrent execution of feature engineering, scaling, clustering, and visualization processes.
- **Graphics (GPU):** Optional NVIDIA GPU with CUDA support (e.g., GTX 1650 or higher) is recommended to accelerate model inference if scaling or real-time predictions are desired.

2. Memory and Storage

- **System RAM:** 8 GB DDR4 minimum; 16 GB recommended for faster processing of large datasets (~500k+ rows).
- **Storage Type:** SSD (Solid State Drive) is essential for rapid loading of Python packages, preprocessed datasets, and saved model artifacts (scaler.pkl, kmeans.pkl).
- **Storage Capacity:** Minimum 256 GB NVMe SSD recommended to handle datasets, logs, segmented outputs, and backups.

3. Input Devices (Optional for Data Upload)

- **File Input:** Standard keyboard and mouse sufficient for dataset uploads (CSV/Excel).
- **External Storage:** USB or network drives may be used for dataset transfer if cloud sync or multi-location data integration is planned.

4. Network and Connectivity

- **Offline Mode (Local):** LAN setup sufficient to deploy Streamlit dashboard and access local datasets securely.
- **Online Mode (Cloud / Remote Access):** Stable internet connection (minimum 5 Mbps) required to allow remote monitoring or uploading datasets from multiple locations.

- **Hybrid Setup:** Hardware must support both LAN and internet access to enable optional cloud-based synchronization of segmented outputs.

5. Display and Interface

- **Monitor:** Full HD display (1080p) recommended to view Streamlit dashboard, tables, and Plotly charts clearly.
- **Peripheral Support:** USB 3.0 or higher recommended for connecting external storage or optional devices for batch dataset import.

4.2 SOFTWARE SPECIFICATIONS

1. Primary Development Stack

- **Programming Language:** Python 3.9+ is the core language, chosen for its extensive support for data science, machine learning, and AI libraries.
- **Web Framework:** Streamlit is used for both frontend and backend, providing a responsive, browser-based dashboard for dataset uploads, segmentation execution, and visualization.
- **Operating Systems:** Fully compatible with Windows 10/11, macOS, and Ubuntu/Linux.

2. Machine Learning & Data Processing Pipeline

- **Data Cleaning & Preprocessing:** Pandas and NumPy handle large datasets, missing values, and feature computations efficiently.
- **Feature Engineering:** Custom Python scripts compute TotalPrice and RFM features to prepare data for clustering.
- **Modeling:** Scikit-Learn's K-Means algorithm performs unsupervised clustering based on RFM features.
- **Scaling:** StandardScaler is applied to normalize features before clustering.
- **Inference Pipeline:** Modular scripts load saved artifacts (scaler.pkl, kmeans.pkl) to predict clusters for new datasets.

3. Data Management & Reporting Framework

- **Data Storage:** Processed datasets, segmented outputs, and model artifacts are stored in structured directories (artifacts/, data/) for reproducibility.
- **Analysis & Reporting:** Pandas and NumPy are used for aggregation, segmentation tables, and automated CSV/Excel exports.
- **Visualization:** Plotly generates interactive charts for segment insights; Streamlit displays them in real-time on the dashboard.
- **Business Segment Mapping:** Cluster IDs are mapped to business labels (High-Value, At-Risk, Recent Low Spenders, Average Buyers) using Python logic modules.

4. Deployment & Infrastructure

- **Deployment Modes:** Supports Online Mode (Cloud hosting for multi-location management) and Offline Mode (LAN or local machine for private, standalone operation).
- **Model Execution Optimization:** Pre-trained artifacts and modular pipelines ensure fast inference (~100–200ms for 500k records) without GPU dependency, though GPU can accelerate scaling.
- **Environment Management:** Python virtual environments or Conda recommended for package consistency and reproducibility.
- **Web Dashboard Integration:** Streamlit frontend communicates with inference pipeline seamlessly, allowing admin login, file upload, visualization, and CSV download in one interface.

CODING AND IMPLEMENTATION

5.1 CODING

```
import streamlit as st
import pandas as pd
import plotly.express as
px import base64
from pathlib import Path

from utils.io import load_customer_file
from inference.batch_predictor import BatchPredictor

#
==================================================================
# PAGE CONFIG
# ==================================================================
st.set_page_config(
    page_title="Retail Customer Segmentation",
    page_icon="🛍️",
    layout="wide"
)

#
==================================================================
# SESSION STATE
# ==================================================================
if "page" not in st.session_state:
    st.session_state.page =
    "login"

if "logged_in" not in st.session_state:
    st.session_state.logged_in = False

#
==================================================================
# THEME
# ==================================================================
def apply_theme():
```

```
st.markdown("""
<style>
html, body, [class*="css"] { color: white !important; }
h1,h2,h3,h4,h5,h6,p,span,label,div { color: white !important; }
```

```
    input {
        color: white !important;
        background-color: #1e1e1e !important;
    }

    button:not(div.modebar button) {
        background-color: black
        !important; color: white !important;
        border: 1px solid white
        !important; border-radius: 999px
        !important; padding: 10px 22px
        !important;
    }

    .stApp { background-color: #0e1117; }
    </style>
    """, unsafe_allow_html=True)

apply_theme()

# ==============================================================
# BACKGROUND
# ==============================================================
def set_background(image_path: str, dim: float = 0.6):
    p = Path(image_path)
    if not p.exists():
        return
    encoded = base64.b64encode(p.read_bytes()).decode()
    st.markdown(f"""
    <style>
    .stApp {{
        background: url("{encoded}") no-repeat center center
        fixed; background-size: cover;
    }}
    .stApp::before {{
```

content: "";

position:

fixed;

```
        inset: 0;
        background:
        rgba(0,0,0,{dim}); z-index: 0;
    }}
    .block-container {{
        position:
        relative; z-index:
        1;
    }}
    </style>
    """, unsafe_allow_html=True)

#
==================================================================
# NAVIGATION
# ==================================================================
def nav_bar():
    col1, col2, col3 = st.columns([4,1,1])

    with col1:
        if st.button("🏠 Home"):
            st.session_state.page = "home"
            st.rerun()
        if st.button("📊 Predict"):
            st.session_state.page = "predict"
            st.rerun()
        if st.button("🔮 Simulator"):
            st.session_state.page = "simulator"
            st.rerun()

    with col3:
        if st.button("🚪 Logout"):
            st.session_state.logged_in = False
            st.session_state.page = "login"
            st.rerun()
```

#

==========

LOGIN PAGE

==========

```
def login_page():
    set_background("bgl.jpg",0.4)
    st.markdown("<h1 style='text-align:center;'>🔐 Admin Login</h1>", unsafe_allow_html=True)

    with st.form("login"):
        u = st.text_input("Username")
        p = st.text_input("Password",
        type="password") if
        st.form_submit_button("Login"):
            if u == "admin" and p == "12345":
                st.session_state.logged_in = True
                st.session_state.page = "home"
                st.rerun()
            else:
                st.error("Invalid credentials")

#
================================================================
# HOME PAGE
# ================================================================
def home_page():
    set_background("bgh.jpg",0.3)
    nav_bar()
    st.markdown("""
    <h1 style="text-align:center; margin-top:120px;">
        🛍️ AI-Powered Customer Intelligence System
    </h1>
    """, unsafe_allow_html=True)

#
================================================================
# PREDICT PAGE
# ================================================================
def predict_page():
    set_background("bgc.jpg",0.6
    ) nav_bar()
```

```
st.markdown("<h2 style='text-align:center;'>📊 Portfolio-Level Segmentation Analysis</h2>", unsafe_allow_html=True)
```

```
uploaded_file = st.file_uploader("Upload Customer Transaction Dataset", type=["csv","xlsx","xls"])

if uploaded_file is None:
    return

raw_df = load_customer_file(uploaded_file)

if st.button("🚀 Run Segmentation Analysis"):
    with st.spinner("Processing dataset..."):
        predictor = BatchPredictor()
        results_df =
        predictor.predict(raw_df)

    st.success("Segmentation                    Completed

    Successfully!") st.dataframe(results_df.head())

    # Segment Distribution
    st.markdown("## 📈 Customer Distribution
    Overview") segment_counts =
    results_df["Segment"].value_counts()
    st.write(segment_counts)

    fig = px.pie(
        values=segment_counts.values,
        names=segment_counts.index,
        title="Segment Composition"
    )
    st.plotly_chart(fig)

    # Business Recommendations
    st.markdown("## 💼 Business Strategy Insights")

    segments_present = segment_counts.index.tolist()

    if "High Value Loyal" in segments_present:
        st.info("""
        **High Value Customers Strategy**
```

- Launch exclusive premium membership

```
        - Offer early product access
        - Provide VIP loyalty incentives

        Objective: Strengthen retention & maximize lifetime revenue.
        """)

    if "Loyal Customers" in segments_present:
        st.info("""
        **Loyal Customer Growth Plan**
        - Introduce cashback incentives
        - Bundle pricing strategies
        - Double loyalty reward points

        Objective: Upgrade to premium tier.
        """)

    if "At Risk Customers" in segments_present:
        st.warning("""
        **Churn Prevention Strategy**
        - Time-sensitive discount offers
        - Personalized re-engagement campaigns
        - Customer feedback surveys

        Objective: Recover revenue and prevent attrition.
        """)

    if "New Customers" in segments_present:
        st.success("""
        **New Customer Acceleration Plan**
        - Welcome promotions
        - Personalized onboarding emails
        - Product discovery campaigns

        Objective: Convert into repeat purchasers quickly.
        """)

# ==========================================================================
```

```
# SIMULATOR PAGE (Refined Language)
# ==============================================================
def simulator_page():
    set_background("bgc.jpg",0.6
    ) nav_bar()

    st.markdown("<h2 style='text-align:center;'>🔮 Individual Customer Scenario Simulator</h2>",
unsafe_allow_html=True)

    col1, col2 = st.columns(2)

    with col1:
        recency = st.number_input("Days Since Last Purchase", min_value=0, value=30)
        frequency = st.number_input("Total Transactions", min_value=1, value=5)
        monetary = st.number_input("Total Revenue Generated (₹)", min_value=0.0, value=1000.0)
        total_quantity = st.number_input("Total Units Purchased", min_value=0, value=20)
        unique_products = st.number_input("Distinct Products Purchased", min_value=0, value=5)

        predict_btn = st.button("🔍 Evaluate Customer Profile")

    with col2:
        st.markdown("""
        ### 📘 Input Guidance
        - Recency reflects engagement freshness
        - Frequency measures purchasing consistency
        - Monetary represents revenue contribution
        - Quantity indicates purchase volume
        -         Unique products show buying
        diversity """)

    if predict_btn:
        with st.spinner("Evaluating customer profile..."):
            predictor = BatchPredictor()

            input_df = pd.DataFrame({
                "Recency": [recency],
                "Frequency":
```

[frequency],

```
        "Monetary": [monetary],
        "TotalQuantity": [total_quantity],
        "UniqueProducts":
        [unique_products]
    })

    scaled = predictor.scaler.transform(input_df)
    cluster = predictor.model.predict(scaled)[0]

segment_map = {
    0: "High Value Loyal",
    1: "Loyal Customers",
    2: "At Risk
    Customers", 3: "New
    Customers"
}

segment = segment_map.get(cluster, "Unidentified Segment")

st.success("Customer Evaluation Complete")

st.markdown(f"""
### 🎯 Customer Classification
**Segment Identified:** {segment}
""")

st.markdown("## 📌 Recommended Strategic Action")

if segment == "High Value Loyal":
    st.info("""
    Provide premium-tier privileges including early product releases,
    priority customer support, and loyalty-exclusive benefits.

    Business Goal: Preserve high revenue contributors and expand lifetime value.
    """)

elif segment == "Loyal Customers":
```

st.info("""

Encourage progression to premium tier through targeted incentives,

```
        bundled offerings, and enhanced reward structures.

        Business Goal: Increase spending intensity and engagement frequency.
        """)

    elif segment == "At Risk Customers":
        st.warning("""
        Deploy reactivation initiatives such as personalized
        outreach, limited-time discounts, and satisfaction surveys.

        Business Goal: Reduce churn probability and recover declining
        revenue. """)

    elif segment == "New Customers":
        st.success("""
        Implement onboarding programs, introductory promotions,
        and guided product recommendations.

        Business Goal: Accelerate conversion into repeat loyal customers.
        """)

# ============================================================
# ROUTER
# ============================================================
if not st.session_state.logged_in:
    login_page()
elif st.session_state.page == "home":
    home_page()
elif st.session_state.page == "predict":
    predict_page()
elif st.session_state.page == "simulator":
    simulator_page()
else:
    home_page()
```

5.2 TESTING

The testing phase of the AI-Powered Customer Segmentation System involves a multi-layered evaluation to ensure accurate, reliable customer segmentation and seamless operation of the Streamlit dashboard. The process begins with unit testing of individual modules such as data preprocessing, RFM feature computation, and clustering, followed by integration testing to verify the complete data flow between dataset upload, preprocessing, ML inference, and dashboard visualization. System testing is conducted to assess end-to-end performance under diverse dataset scenarios, including missing CustomerIDs, cancellations, and large-scale datasets (~500k rows), while acceptance testing validates that the dashboard functions correctly for administrative users in both online and offline environments. Final verification ensures that segmented outputs, business insights, and downloadable CSV files are generated accurately and consistently.

5.2.1 UNIT TESTING

Unit testing focuses on individual components or “units” of code to ensure correct functionality in isolation.

- **Dataset Upload & Validation:** Verify that the system accepts CSV/Excel uploads and correctly flags unsupported formats.
- **Data Cleaning Module:** Test functions for removing cancellations and handling missing CustomerIDs; ensure negative Quantity values are properly excluded.
- **Feature Engineering:** Pass sample transactional data to ensure TotalPrice and RFM features are computed accurately for each customer.
- **Scaling Module:** Verify that StandardScaler transforms input features correctly without altering dataset shape.
- **Clustering Module (K-Means):** Input sample RFM features and confirm that cluster labels are generated as expected.
- **Business Segment Mapping:** Test mapping logic to ensure cluster IDs correspond to correct business labels (High-Value, At-Risk, Recent Low Spenders, Average Buyers).

5.2.2 INTEGRATION TESTING

Integration testing evaluates how different modules work together as a group.

- **Upload-to-Preprocessing Pipeline:** Ensure uploaded datasets pass correctly from the Streamlit upload module to the preprocessing functions without data loss.
- **Preprocessing-to-ML Pipeline:** Verify that cleaned, scaled RFM features are correctly passed into the K-Means clustering module.
- **Artifact Loading:** Confirm that scaler.pkl and kmeans.pkl are correctly loaded and applied during inference for consistent predictions.
- **Dashboard Integration:** Test that clicking "Run Segmentation" on Streamlit triggers the full backend pipeline and updates the dashboard in real-time.
- **CSV Export:** Verify that segmented customer data and mapped business labels are written correctly to downloadable CSV files.

5.2.3 SYSTEM TESTING

System testing validates the complete, integrated software to ensure it meets specifications under realistic conditions.

- **Accuracy & Consistency:** Test with multiple datasets of varying sizes and distributions to confirm correct cluster formation and business segment mapping.
- **Performance & Latency:** Measure total "Upload-to-Result" time to ensure the system handles large datasets (~500k rows) efficiently without crashes.
- **Edge Cases:**
    - Missing CustomerIDs: Confirm these are excluded or handled gracefully.
    - Negative quantities: Confirm returns are excluded from aggregations.
    - Mixed file formats: Ensure unsupported formats are rejected.
- **Dashboard Robustness:** Test all visualization components (Plotly charts, tables) under heavy loads and ensure interactive features work correctly.

5.2.4 ACCEPTANCE TESTING (ONLINE / OFFLINE MODE)

- **Offline Mode Test:** Verify that the system works on a local machine or LAN, ensuring all preprocessing, clustering, and visualizations function without internet connectivity.
- **Online Mode Test:** Verify that administrators can upload datasets, run segmentation, view insights, and download CSV reports from remote cloud deployment.
- **User Acceptance:** Confirm that end-to-end workflow meets business requirements, including identification of high-value, at-risk, and average customers, with correct visualization and export functionality.

SECURITY,BACKUPAND RECOVERY MECHANISM

5.1 SECURITY, BACKUP AND RECOVERY MECHANISM

Security:

Security in the AI-Powered Customer Segmentation System is designed to protect sensitive business data and maintain privacy by design. Customer transactional data, RFM features, and cluster assignments are stored and processed securely, with strict access controls implemented in the Streamlit dashboard to ensure that only authorized administrators or business analysts can upload datasets, view insights, or download segmented CSV files.

Sensitive artifacts, such as `scaler.pkl` and `kmeans.pkl`, are stored in protected directories, and file permissions are enforced to prevent unauthorized access or tampering. All data in transit between the front-end dashboard and backend pipeline is secured via HTTPS (for cloud deployments) or LAN encryption protocols (for offline/private deployments). Optional encryption of CSV exports ensures that sensitive customer insights cannot be intercepted or misused.

Backup:

To ensure the integrity and availability of processed datasets, feature-engineered data, model artifacts, and segmented outputs, a robust **backup strategy** is implemented.

- **3-2-1 Backup Rule:**
  - Three copies of critical data: primary working directory, local external backup (HDD/SSD), and off-site cloud storage.
- **Backup Frequency:**
  - Full backups of all artifacts (scaler.pkl, kmeans.pkl, cleaned datasets, segmented CSVs) are performed weekly.
  - Incremental backups capture new segmentation outputs and updates to transactional datasets daily.
- **Purpose:**

- Ensures that all processed data, ML artifacts, and business insights remain recoverable in the event of hardware failure, accidental deletion, or cyber threats.

Recovery:

The recovery mechanism focuses on minimizing downtime and preventing data loss.

- **Point-in-Time Recovery:** Allows restoration of datasets and model artifacts to a specific state prior to a failure.
- **Offline Mode:** In local deployments, the Streamlit dashboard and ML pipeline continue functioning on a private LAN even if the primary cloud connection fails.
- **Testing & Validation:** Regular recovery drills are conducted by simulating system failures to confirm that backups are complete, valid, and can be reloaded seamlessly into the pipeline without affecting the accuracy or integrity of customer segmentation results.
- **Objective:** Minimize business disruption while preserving consistency of RFM computations, cluster assignments, and business insights.

FUTURE ENHANCEMENT

5.2 Future Enhancement

Future enhancements for the AI-Powered Customer Segmentation System focus on improving predictive intelligence, scalability, security, and integration with enterprise business processes. Based on industry trends in customer analytics and unsupervised learning systems, here are the key areas for future scope:

1. Advanced Analytics & Predictive Insights

- **Predictive Customer Behavior:** Integrate machine learning models to forecast future spending, churn risk, or loyalty trends beyond static RFM segmentation.
- **Anomaly Detection:** Implement algorithms to detect unusual purchasing patterns, such as sudden spikes or drops in order frequency, to flag potential fraud or at-risk customers.
- **Cross-Sell & Up-Sell Recommendations:** Use cluster insights combined with market basket analysis to suggest products to high-value customers automatically.

2. System Scalability & Performance

- **Distributed Computing:** Implement Spark or Dask for preprocessing and clustering of very large datasets (millions of transactions) to reduce processing time.
- **GPU Acceleration:** Use GPUs for feature scaling and K-Means inference for faster real-time predictions on large datasets.
- **Cloud Deployment & Multi-Tenant Support:** Enable deployment across multiple branches or regions, allowing centralized analytics with secure data segregation.

3. Security & Data Privacy Enhancements

- **Encryption at Rest & Transit:** Advanced encryption (AES-256 / TLS 1.3) for all stored datasets, artifacts, and CSV exports.
- **Role-Based Access Control (RBAC):** Granular permissions for administrators, analysts, or auditors to view or modify only authorized modules.
- **Audit Logs:** Maintain a secure log of dataset uploads, segmentation runs, and CSV downloads for compliance and traceability.

4. User Experience & Interface Improvements

- **Interactive Dashboard Enhancements:** Add drill-down analytics, filtering, and dynamic visualizations for individual customer clusters.
- **Mobile-Friendly Dashboard:** Optimize Streamlit interface for tablets or mobile devices for remote monitoring.
- **Automated Reporting:** Schedule automatic CSV exports and email notifications to stakeholders with insights and segmented results.

5. Integration & Multi-Source Data Support

- **ERP / CRM Integration:** Seamlessly feed segmentation results into existing enterprise systems to inform marketing, sales, or loyalty programs.
- **Multi-Channel Data Ingestion:** Include e-commerce, POS, and online behavior data for richer feature engineering beyond traditional transactional data.
- **API-First Architecture:** Enable programmatic access to segmentation results, supporting third-party analytics or BI tools.

SOFTWARE MAINTENANCE

5.5 Software Maintenance

Software maintenance for the AI-Powered Customer Segmentation System is the ongoing process of updating, optimizing, and improving the system after initial deployment. Since the system relies on transactional data preprocessing, RFM computations, and K-Means clustering, maintenance is critical to ensure accurate segmentation, consistent dashboard functionality, and reliable CSV exports over time.

Here is the maintenance plan structured according to the four standard types:

1. Corrective Maintenance (Bug Fixing)

- **Logical Errors:** Resolve issues where cluster assignments or business segment mappings may be incorrect due to data anomalies or rounding errors in RFM calculations.
- **Data Pipeline Failures:** Fix errors if the preprocessing pipeline fails on unusual datasets, e.g., missing CustomerIDs, negative quantities, or corrupted CSV files.
- **Library Conflicts:** Patch broken dependencies or deprecated functions in Python libraries like Pandas, NumPy, or Scikit-Learn that may disrupt feature engineering or clustering.

2. Adaptive Maintenance (Environment Sync)

- **Operating System Updates:** Ensure compatibility with new versions of Windows, macOS, or Linux, updating Python environment and Streamlit framework as needed.
- **Deployment Environment Changes:** Adjust the pipeline logic if cloud provider settings, network protocols, or file storage locations are modified.
- **Regulatory Compliance:** Update data handling and storage mechanisms to comply with new privacy regulations (e.g., GDPR, CCPA), especially for customer transactional data.

3. Perfective Maintenance (Performance Tuning)

- **Model Re-Calibration:** Periodically re-run K-Means on updated datasets to reflect changing customer behavior, maintaining meaningful cluster segmentation.
- **Scaling Optimization:** Adjust feature scaling and clustering parameters for performance improvements as dataset size grows.
- **UI/UX Enhancements:** Improve Streamlit dashboard interactivity, visualizations, or reporting features based on user feedback from business analysts.

4. Preventive Maintenance (Future-Proofing)

- **Code Refactoring:** Optimize and modularize Python scripts to reduce technical debt and simplify future upgrades.
- **Dependency Updates & Security:** Proactively update Python libraries and dependencies to prevent vulnerabilities or incompatibilities.
- **Monitoring & Alerts:** Implement automated logging of preprocessing time, clustering runtime, and dashboard responsiveness; trigger notifications if performance degrades.

CONCLUSION

5.6 CONCLUSION

The AI-Powered Customer Segmentation System represents a transformative approach to retail analytics, moving beyond traditional manual reporting and simple descriptive statistics toward a fully automated, intelligent customer intelligence ecosystem. By leveraging RFM-based feature engineering and K-Means clustering, the system delivers actionable insights into customer behavior, enabling businesses to identify high-value loyal customers, at-risk segments, recent low spenders, and average buyers with precision. This segmentation empowers targeted marketing, personalized promotions, and data-driven decision-making.

Technically, the system excels through its modular, production-ready pipeline, which handles large-scale transactional datasets (~500k rows) efficiently. The integration of scalable preprocessing, feature scaling, and artifact-based inference ensures rapid cluster assignment, allowing administrators to generate segmented outputs and business insights in near real-time. The Streamlit dashboard provides an intuitive interface for uploading datasets, running segmentation, visualizing results with Plotly charts, and downloading reports—all while maintaining data integrity and consistency across multiple sessions.

One of the system's key advantages is its hybrid deployment model, supporting both offline (LAN-based) and online (cloud-hosted) operation. This flexibility ensures that sensitive customer data remains secure and under organizational control while allowing multi-location access for analytics and reporting. The architecture separates training and inference pipelines, preserving the stability of production deployments and enabling future enhancements without disrupting business operations.

Ultimately, this project is a comprehensive solution that balances technical sophistication with business usability. It provides actionable insights in a secure and scalable manner, while the modular design allows for expansion into advanced analytics, predictive modeling, and multi-source data integration. As the system evolves to incorporate predictive behavior modeling, automated reporting, and enterprise integrations, it sets a solid foundation for the next generation of intelligent, data-driven customer engagement strategies.

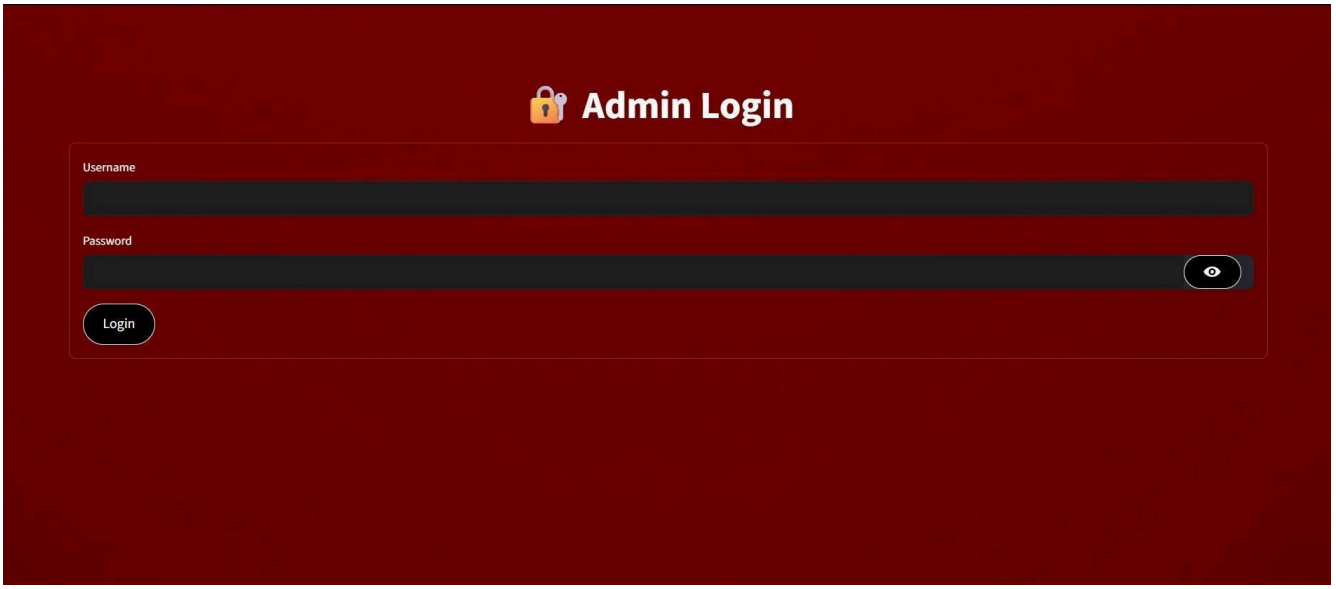
APPENDIX

GANTT CHART

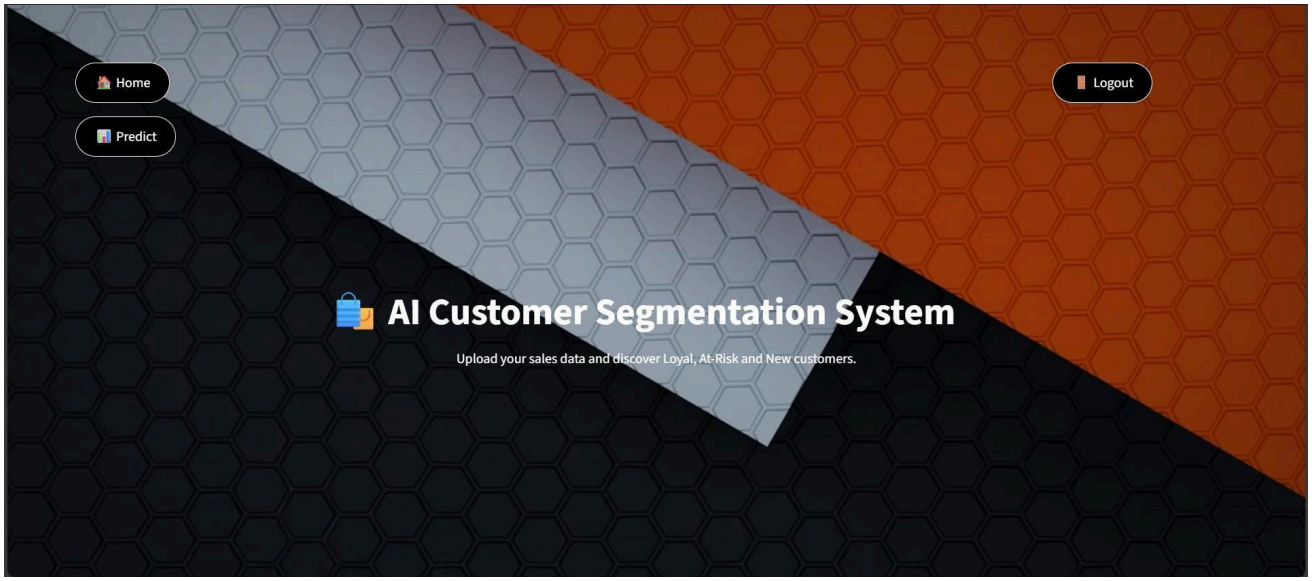
Dates	Oct 8	Dec 7-17	Dec 2-13	Jan 2-8	Jan 14	Jan 16-20	Feb 2	Feb 7	Feb 11
Planning and Analysis									
System Design									
Coding									
System Testing									
Installation									
User Training									
Documentation									

SCREENSHOTS

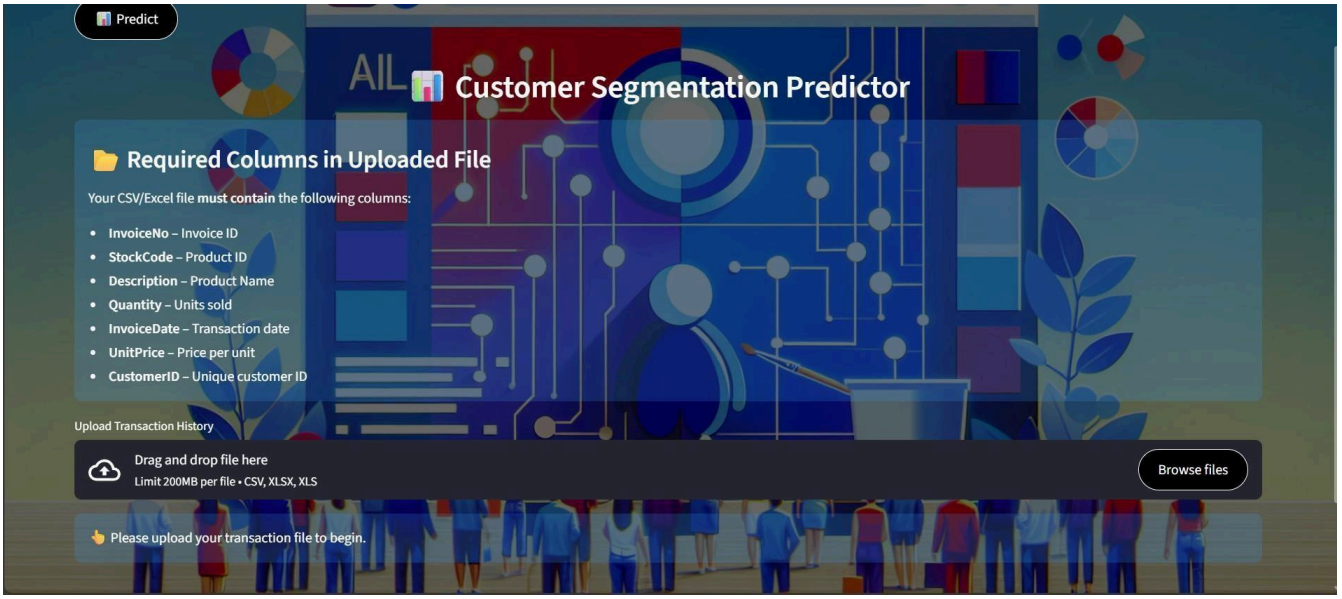
ADMIN LOGIN



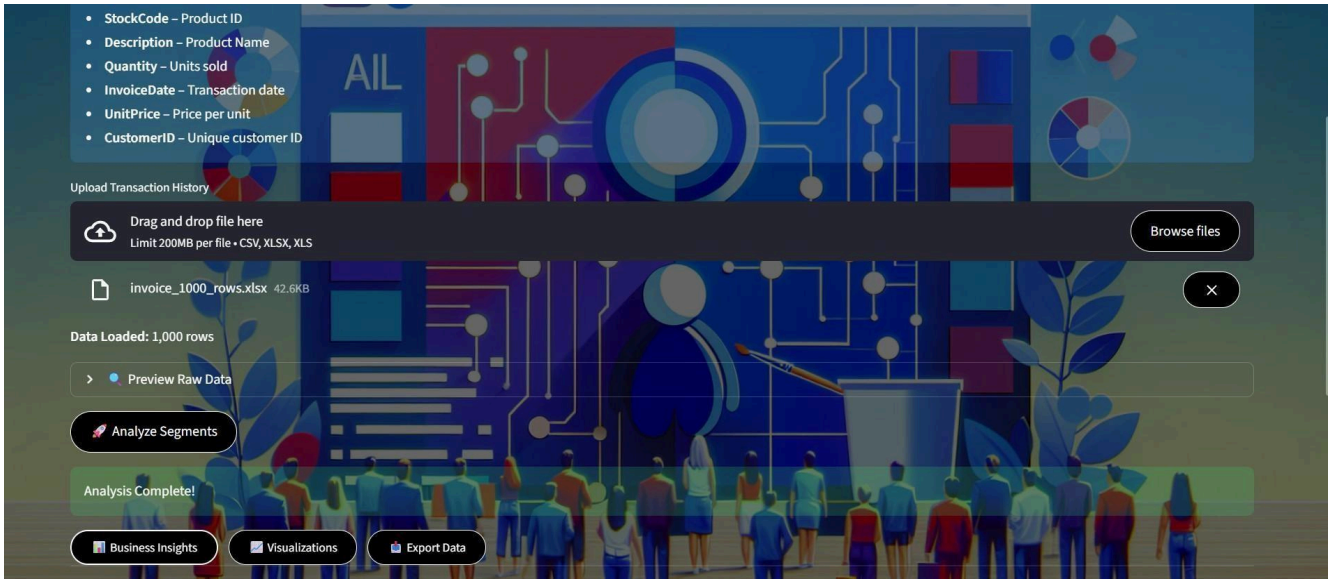
HOME PAGE



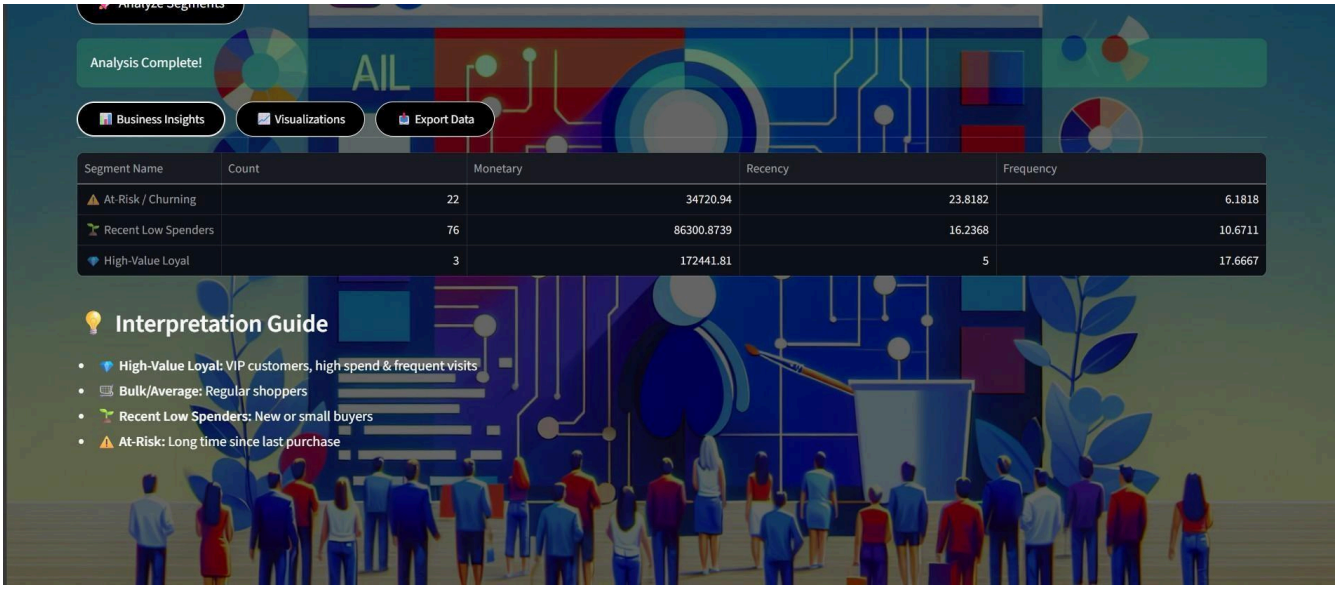
PREDICT PAGE



RAW DATA INSERTED



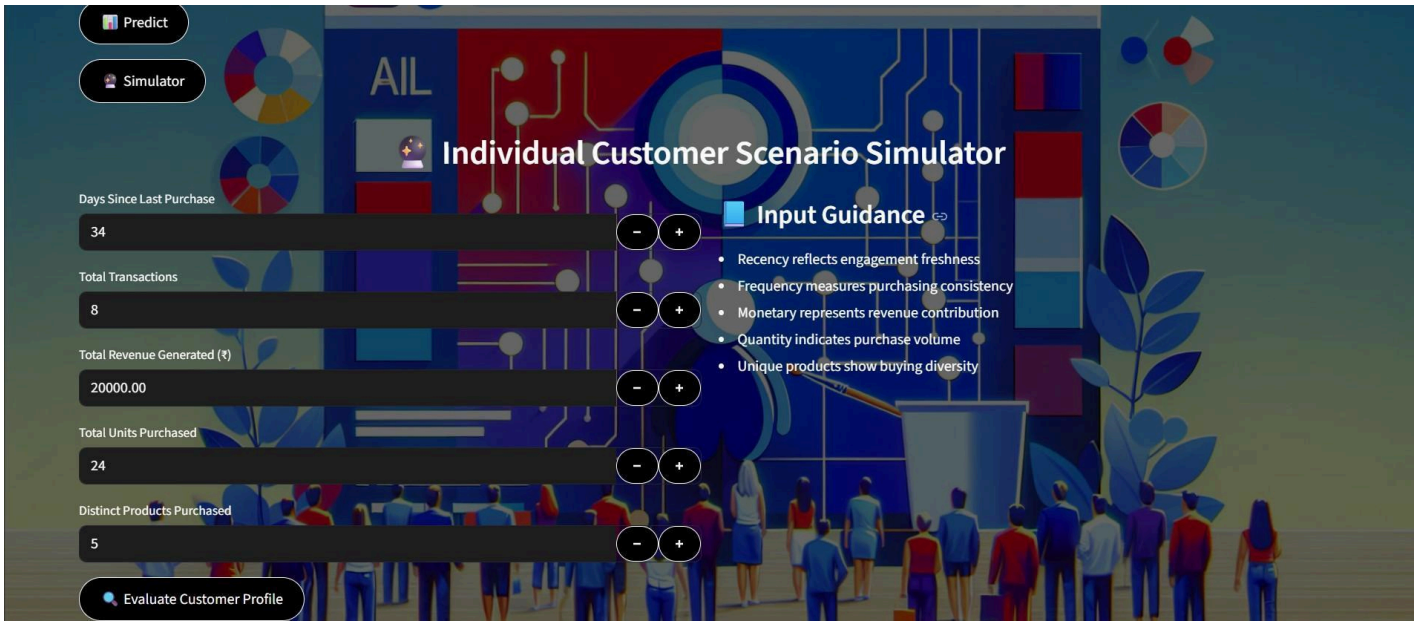
BUSINESS INSIGHT OUTPUT



Segment Name	Count	Monetary	Recency	Frequency
At-Risk / Churning	22	34720.94	23.8182	6.1818
Recent Low Spenders	76	86300.8739	16.2368	10.6711
High-Value Loyal	3	172441.81	5	17.6667

OUTPUT VISUALISATION

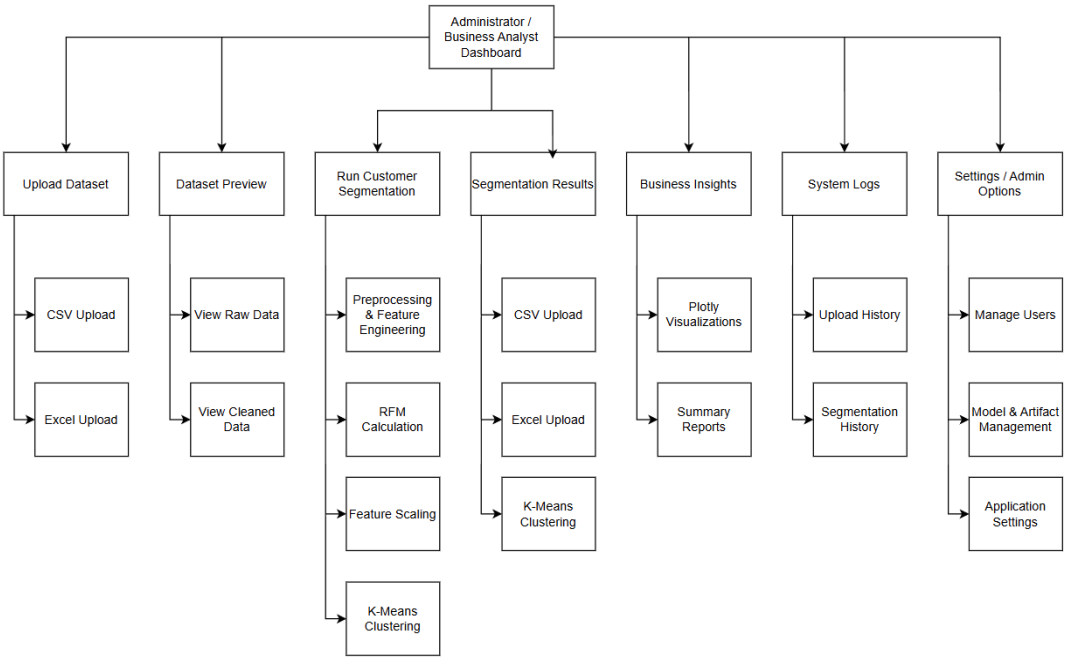
CUSTOMER SIMULATER



EXPORTED DATA

	A	B	C	D	E	F	G	H
1	CustomerI	Recency	Frequency	Monetary	TotalQuan	UniquePro	Cluster	Segment Name
2	1000	10	14	93681.67	138	7	3	ðŸŒ± Recent Low Spenders
3	1001	5	9	91321.73	106	7	3	ðŸŒ± Recent Low Spenders
4	1002	31	11	45426.77	66	7	3	ðŸŒ± Recent Low Spenders
5	1003	15	9	63894.67	86	7	3	ðŸŒ± Recent Low Spenders
6	1004	3	8	59969.58	102	5	3	ðŸŒ± Recent Low Spenders
7	1005	19	12	100593.2	132	7	3	ðŸŒ± Recent Low Spenders
8	1006	7	8	53887.43	76	7	3	ðŸŒ± Recent Low Spenders
9	1007	10	19	178449.7	188	6	1	ðŸ'Ž High-Value Loyal
10	1008	12	10	62258.58	88	6	3	ðŸŒ± Recent Low Spenders
11	1009	4	5	19466.78	44	4	0	âš ï¸ At-Risk / Churning
12	1010	19	10	86390.54	121	6	3	ðŸŒ± Recent Low Spenders
13	1011	9	10	68750.4	97	7	3	ðŸŒ± Recent Low Spenders
14	1012	64	8	67171.1	75	4	3	ðŸŒ± Recent Low Spenders
15	1013	5	13	96400.96	139	7	3	ðŸŒ± Recent Low Spenders
16	1014	14	5	33866.67	33	3	0	âš ï¸ At-Risk / Churning
17	1015	15	14	135153.7	155	6	3	ðŸŒ± Recent Low Spenders
18	1016	37	10	66004.67	80	7	3	ðŸŒ± Recent Low Spenders
19	1017	9	11	96072.96	120	7	3	ðŸŒ± Recent Low Spenders
20	1018	15	7	49164.75	89	5	3	ðŸŒ± Recent Low Spenders
21	1019	2	15	66647.3	117	9	3	ðŸŒ± Recent Low Spenders
22	1020	26	12	87729.64	118	8	3	ðŸŒ± Recent Low Spenders
23	1021	6	14	136154.2	169	7	3	ðŸŒ± Recent Low Spenders
24	1022	13	14	154230.3	163	8	3	ðŸŒ± Recent Low Spenders
25	1023	16	13	102918.3	138	7	3	ðŸŒ± Recent Low Spenders
26	1024	2	10	63133.3	114	6	3	ðŸŒ± Recent Low Spenders

MENU TREE



MEETING MINUTES

Date:

08/10/2025

Place:

Keltron

Discussion: Started preparing synopsis.

Members: Abdul Raheem, Ashish Binu, Safwan Yasir, Adhil

Muhammed Date: 17/10/2025

Place: Keltron

Discussion: Started classes of AI

Members: Abdul Raheem, Ashish Binu, Safwan Yasir, Adhil Muhammed

Date: 17/10/2025 Place: Keltron Discussion: Started design of the project Members: Abdul Raheem, Ashish Binu, Safwan Yasir, Adhil Muhammed

Date:

12/12/2025

Place:

Keltron

Discussion: Completed Design

Members: Abdul Raheem, Ashish Binu, Safwan Yasir, Adhil Muhammed

Date: 26/12/2025

Place: Keltron

Discussion: Coding started for the User module.

Members: Abdul Raheem, Ashish Binu, Safwan Yasir, Adhil Muhammed

Date: 31/12/2025

Place: Keltron

Discussion: Completed Coding

Members: Abdul Raheem, Ashish Binu, Safwan Yasir, Adhil Muhammed

Date: 04/01/2026

Place: Library

Discussion: Testing of Each Module

Members: Abdul Raheem, Ashish Binu, Safwan Yasir, Adhil Muhammed

Date: 14/01/2026

Place: Computer Lab

Discussion: Test Validations

Members: Abdul Raheem, Ashish Binu, Safwan Yasir, Adhil Muhammed

Date: 7/02/2026

Place: Keltron

Discussion: Documentation started

Members: Abdul Raheem, Ashish Binu, Safwan Yasir, Adhil Muhammed

Date: 11/02/2026

Place: Library

Discussion: Completed Documentation

Members: Abdul Raheem, Ashish Binu, Safwan Yasir, Adhil Muhammed

BIBLIOGRAPHY

BOOK REFERENCES

- Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media.
- Raschka, S., & Mirjalili, V. (2022). *Python Machine Learning: Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow 2* (4th ed.). Packt Publishing.
- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.
- McKinney, W. (2018). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython* (2nd ed.). O'Reilly Media.

website references

- Scikit-Learn Documentation
- Streamlit Documentation
- Pandas Documentation
- NumPy Documentation
- Plotly Python Documentation
- Jupyter Notebook Documentation